

**SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
VARAŽDIN**

Aleksandar Babić

**INFORMATIVNA PLATFORMA S
INTELIGENTNIM ASISTENTOM O
ERASMUS+ ISKUSTVU U ŁÓDŻU**

DIPLOMSKI RAD

Varaždin, 2025.

SVEUČILIŠTE U ZAGREBU
FAKULTET ORGANIZACIJE I INFORMATIKE
V A R A Ž D I N

Aleksandar Babić

Matični broj: 0016147642

Studij: Baze podataka i baze znanja

**INFORMATIVNA PLATFORMA S INTELIGENTNIM ASISTENTOM O
ERASMUS+ ISKUSTVU U ŁÓDŽU**

DIPLOMSKI RAD

Mentor:

doc. dr. sc. Bogdan Okreša Đurić

Varaždin, rujan 2025.

Zahvala

Na samom početku želim izraziti iskrenu zahvalnost svom mentoru, doc. dr. sc. Bogdanu Okreši Đuriću, na izdvojenom vremenu, nesebičnoj podršci i strpljenju koje mi je pružio tijekom izrade ovog diplomskog rada.

Posebnu zahvalnost dugujem svojoj obitelji – ocu, majci i sestri – za njihovu podršku i ohrabrenje tijekom cijelog mog studija. Njihova prisutnost i vjera u mene davali su mi snagu u trenucima kada je to bilo najpotrebnije.

Na kraju, veliko hvala svim prijateljima koje sam upoznao tijekom studiranja. Zajedno smo prošli kroz brojne izazove, ali i podijelili mnogo smijeha i lijepih trenutaka koje ću zauvijek nositi sa sobom.

Izjava o izvornosti

Izjavljujem da je ovaj diplomski rad izvorni rezultat mojeg rada te da se u izradi istoga nisam koristio drugim izvorima osim onima koji su u njemu navedeni. Za izradu rada su korištene etički prikladne i prihvatljive metode i tehnike rada.

Autor potvrdio prihvatanjem odredbi u sustavu FOI Radovi

Sažetak

Teoretski dio rada istražuje studiranje kao važan segment u životnom razdoblju pojedinca te se dotiče razmjene studenata u svrhu Erasmus+ studijskog boravka. Teoretski dio završava poglavljem posvećenim inteligentnim asistentima temeljenima na RAG (eng. Retrieval-Augmented Generation) tehnologiji. Praktični dio rada obuhvaća izradu web-stranice s tematikom Erasmus+ studijskog boravka i izradu inteligentnog asistenta, pri čemu je iskorišten postojeći jezični model. To podrazumijeva treniranje i prilagođavanje specifičnom skupu podataka, kako bi se osiguralo da inteligentni asistent odgovara upitima korisnika i pruža relevantne informacije.

Ključne riječi: Inteligentni asistent, RAG, web stranica, studiranje, Erasmus+, umjetna inteligencija.

Sadržaj

1. Uvod	1
2. Studiranje kao dio života pojedinca	2
2.1. Odluka i benefiti	2
2.2. Erasmus+ studentska mobilnost	4
2.2.1. Izazovi i strahovi	5
3. Aktualne tehnologije i alati za uspješno snalaženje i informiranje	7
3.1. Važnost inteligentnih pomoćnih sustava	7
3.1.1. Asistent	8
3.1.1.1. Siri	8
3.1.2. Copilot	9
3.1.2.1. Microsoft Copilot Studio	9
3.1.3. Agent	11
3.1.3.1. ChatGPT Operator	11
3.1.4. Chatbot	12
3.1.4.1. Intercom	12
3.2. Inteligentni asistenti u obrazovnom sustavu	13
3.2.1. Primjena inteligentnih asistenata u obrazovanju	13
3.2.2. Izazovi inteligentnih asistenata u obrazovanju	14
4. Retrieval Augmented Generation metoda	15
4.1. Uvod u RAG	15
4.1.1. Povijest i razvoj	16
4.2. Arhitektura RAG modela	18
4.2.1. Faza dohvaćanja (Retriever)	19
4.2.1.1. BM25	20
4.2.1.2. Dense Passage Retrieval	20
4.2.1.3. REALM	20
4.2.2. Faza generiranja (Generator)	21
4.2.2.1. T5	21
4.2.2.2. BART	21
4.2.2.3. GPT-4	21
4.3. Evaluacija i primjena RAG metode	22
4.3.1. Usporedba: RAG vs. Fine-tuning	22
4.3.2. Prednosti RAG sustava	23
4.3.3. Tipični slučajevi upotrebe	24

4.3.4. Izazovi RAG pristupa	25
4.3.4.1. Halucinacije	26
4.3.4.2. Upravljanje višestrukim izvorima	27
4.4. Dostupni alati i biblioteke	29
4.4.1. Okviri (frameworkovi) za upravljanje RAG-om	29
4.4.1.1. Haystack	29
4.4.1.2. LangChain	30
4.4.1.3. LlamaIndex	30
4.4.2. Vektorske baze podataka	31
4.4.2.1. FAISS	33
4.4.2.2. Qdrant	33
4.4.2.3. Pinecone	33
4.4.3. Veliki jezični modeli	33
4.4.3.1. OpenAI	34
4.4.3.2. Hugging Face	34
4.4.4. Ostale tehnologije	35
4.4.4.1. React	35
4.4.4.2. SMTP	36
4.4.4.3. FastAPI	37
5. Praktični dio rada	38
5.1. Struktura web stranice	39
5.2. Prikupljanje podataka	39
5.3. Korisničko sučelje (Frontend)	44
5.4. Poslužiteljski dio (Backend)	49
5.4.1. Server	50
5.4.2. Servisi	50
5.4.3. Rute	54
5.5. Prikaz rada aplikacije	59
5.6. Praćenje performansi i ograničenja korištenih alata	64
6. Zaključak	65
Popis literature	68
Popis slika	70
Popis tablica	71
Popis isječaka koda	73

1. Uvod

Na početku rada obrađuje se pojam studiranja kao i svi aspekti koji se usko vežu uz njega. Poseban naglasak je stavljen na važnost studiranja u kontekstu osobnog razvoja, izgradnje karaktera te stjecanja različitih znanja. U nastavku se detaljnije razmatra Erasmus+ program, s ciljem približavanja njegovih mogućnosti i koristi. Glavni cilj je ukazati na izazove poput nedostatka interesa za prijavu na Erasmus+ studijski boravak, prisutnog straha, nesigurnosti te neinformiranosti među studentima.

Kao rješenje na navedene izazove izrađena je web stranica s inteligentnim asistentom koji je namijenjen da odgovori na specifične upite korisnika. Web stranica predstavlja sredstvo informiranja prvotno za sve studente koji dolaze u Łódź iz svojih država. Osim toga web stranica je naravno dostupna i namijenjena i za sve ostale zainteresirane posjetitelje. Web stranica sastoji se od sljedećih dijelova: početne stranice, stranice s kratkim opisom grada Łódź, stranice koja prikazuje 15 najvećih gradova Poljske, stranice za pregled i upravljanje dokumentima korištenima u RAG sustavu, te stranice s kontaktnom formom. Forma je namijenjena za pitanja na koja AI (eng. Artificial Intelligence) asistent nije znao odgovoriti.

Za izradu AI modela odabrano je korištenje RAG metode (eng. Retrieval Augmenting Generation) gdje se uzima postojeći AI model i on se bazira na specifičnom skupu podataka. Za takve potrebe pronađeno je nekoliko lokalnih web stranica koje opisuju život u Łódźu. Za potrebe rada razvijen je scraper koji dohvaća primarno tekstualni sadržaj s istih tih stranica i sprema ih u JSON obliku. Naknadno su te JSON datoteke pročišćene i iskorištene za daljnje korištenje. Tijekom svoje Erasmus+ studentske mobilnosti imao sam prilike razgovarati s profesoricom na kolegiju Umjetna inteligencija, kojoj sam ukratko predstavio ideju svog diplomskog rada. Naknadno sam je zamolio da mi preporuči nekoliko korisnih stranica koje su vezane za Łódź. Budući da se profesorica rodila u Łódźu i tamo živi te radi cijeli život, smatrao sam da je upravo ona prava osoba kojoj se mogu obratiti s pitanjem o korisnim izvorima informacija vezanima za život u Łódźu.

Za izradu rješenja korištene su sljedeće tehnologije: React (JavaScript) za frontend, Python za razvoj scrapera te Python FastAPI za implementaciju backend logike. AI asistent izrađen je korištenjem tehnologije LangChain, uz model llama-3.1-8b-instant koji je integriran putem API ključa s platforme Groq. Za generiranje vektorskih reprezentacija korišten je HuggingFaceEndpointEmbeddings, također preko API ključa s platforme Hugging Face. Kao vektorska baza podataka korišten je FAISS (Facebook AI Similarity Search), dok je dokumentacija pisana u LaTeX-u, koristeći distribuciju MiKTeX.

Prethodno ulomci uvode nas u temu s kojom se bavi ovaj rad, a to je „Informativna platforma s inteligentnim asistentom o Erasmus+ iskustvu u Łódźu“. Prvotna motivacija za proučavanje ove teme proizišla je iz Erasmus+ studijskog boravka u Łódźu (Poljska) u trajanju jednog semestra.

2. Studiranje kao dio života pojedinca

Važnost učenja se proteže kroz sve uzraste i faze života. Već od malih nogu školska djeca se potiču da marljivo uče jer to na neki način može odrediti njihov put u budućnosti. Unutar ovog poglavlja više je vremena posvećeno studiranju kao važnom segmentu u životnom razdoblju pojedinca. U tom kontekstu visoko obrazovanje može napraviti razliku između uspješne osobe i neuspjeha. Ulaganje u znanje je beskrajno i nikada ne prestaje i može se razmotriti kao investicija u budućnost. Život bez obrazovanja je kao tijelo bez duše, jer obrazovanje pojedincu donosi smisao, znanje i prilike koje oblikuju njegov osobni i profesionalni razvoj. Pomaže u razvoju kritičkog mišljenja, ima plemenit karakter i razvija vještine koje su potrebne za suočavanje sa svim životnim izazovima. Prema riječima stručnjaka iz škola u Bengaluru (Indija), studij je važan jer se koristi za ublažavanje kompleksnosti većine izazova s kojim se susretnu pojedinac [1].

2.1. Odluka i benefiti

Kod donošenja velikih odluka kao što je početak studiranja vrlo je bitno uzeti u obzir nekoliko čimbenika koji mogu oblikovati bolji pogled na razmišljanje o nečemu. Kada se razmišlja o tome da li krenuti studirati ili ne, postoje ljudi koji govore da upisivanje fakulteta više nije presudno ni važno za uspjeh ili da se može dobiti dobar posao bez odlaska na fakultet. Primarni cilj ove sekcije nije zastupanje određene strane, već pružanje uvida u razloge zbog kojih bi upis na fakultet mogao biti dobra odluka. Započetak fakultet priprema radnu snagu za budući posao i može dati značajnu prednost prilikom prijave za posao. Fakultetsko obrazovanje pokazuje poslodavcima da je osoba sposobna dovršiti dugoročni projekt, da je sposobna kritički razmišljati, rješavati probleme i da je sposobna učiti nove stvari. Većina poslova danas zahtijeva barem neko fakultetsko iskustvo i vjerojatno će osobe bez diplome biti u nepovoljnijoj situaciji u usporedbi s konkurencijom. Čak i ako se ne odluči upisati fakultet postoje razni programi, certifikati i tečajevi koji mogu pomoći u razvoju vještina i znanja koje su potrebne za radno mjesto [2].

Umrežavanje je možda i najveći benefit koji se može steći tijekom studiranja. Upoznavanje novih ljudi i gradnja važnih odnosa koji mogu potrajati dugo u budućnosti. Fakultet je mjesto koje nudi priliku upoznavanja drugih studenata s istim/različitim interesima a osim toga postoje i prilike umrežavanja s profesorima i drugim stručnjacima. S toliko studenata i članova fakulteta iz različitih geografskih sredina, sigurno će se pronaći neko tko dijeli iste stavove. Pronaći će se kolega za učenje, kavu ili osoba koja će postati najbolji prijatelj. Osim toga unutar studentskih kampusa moguće je upoznavanje velikog broja ljudi kroz razna sportska događanja. Kroz upoznavanja, osoba može puno naučiti o sebi i onome što ju zanima. Moguće je upoznavanje s osobama koje dolaze iz svih krajeva svijeta čime se mogu okusiti nove kulture i perspektive. Možda će se pojaviti prilika za život sa studentom/studenticom iz druge zemlje ili će se možda pojaviti profesor koji predaje s međunarodnim iskustvom. Takva iskustva mogu razviti bolje razumijevanje svijeta oko sebe. Pohađanje sajмова karijera na fakultetima ili pridruživanje profesionalnim organizacijama su također neke od prilika koje mogu osnažiti i

izgraditi mrežu koja može obogatiti buduću karijeru [2].

Mnogi ljudi vjeruju da je primarna svrha fakulteta pripremiti osobu za određenu karijeru. Fakulteti općenito nude nekoliko programa koji se mogu upisati, ali fakultetsko obrazovanje je više od "samog dobivanja posla". Jedna od važnijih stvari je razvoj kritičkog razmišljanja i rješavanja problema. U stvarnom svijetu vjerojatno će biti situacija kada će osoba ostati sama i u tom trenutku mora biti sposobna shvatiti što je problem i kako ga riješiti [2].

Na današnjem "konkurentnom tržištu rada", fakultetska diploma može dati prednost koja je potrebna kako bi se osoba izdvojila u moru ostalih kandidata. Iako je za dosta poslova dovoljna srednjoškolska diploma, poslodavci često preferiraju kandidate s fakultetskim obrazovanjem. U zadnje vrijeme mnogo fakulteta nudi i zahtjeva stažiranje ili obavljanje nekog oblika prakse čime se zapravo pruža prilika osobi dati uvid u buduću profesiju [2].

Svaka industrija više-manje zahtjeva da osoba bude u toku s trendovima. Ako je prisutna svjesnost o najnovijima dostignućima i osim toga ako postoji šansa da se primjene u radu time je zapravo pokazana želja za povećanjem produktivnosti i učinkovitosti. Praćenje trendova obično radi razliku i privlači tržište. Na fakultetima su profesori većinom dobro povezani i informirani o najnovijim trendovima u industriji [2].

Fakultet pruža pristup vrijednim resursima tijekom studiranja pa i nakon diplomiranja. Jedan od primjera su sveučilišne knjižnice koje često nude širok izbor knjiga i drugih materijala. Također nude pomoć u istraživanju od strane knjižničara koji su stručnjaci u pronalaženju potrebnih informacija. Centar karijera je još jedno vrijedno sredstvo koje može pomoći u domenu poput pisanja životopisa pa do vježbanja razgovora za posao [2].

Pojedini ljudi odlaze na fakultet iz razloga jer do tog trenutka još ne znaju što će biti njihov posao i kako će izgledati njihova karijera. U tom slučaju fakultet može biti korisna odskočna daska ako osoba nije sigurna što želi raditi sa svojim životom. Istraživanjem različitih područja studija može se više saznati o svojim interesima i samim time prepoznati koja područja su vrijedna daljnjeg istraživanja. I onda naknadno to može biti izvrsna prilika za istraživanje ili specijalizaciju područja koje je predmet interesa [2].

Osim toga što fakultet nudi određeni oblik akademskog učenja, on također nudi priliku razvoja osobnih i profesionalnih vještina. Na primjer, učinkovite komunikacijske vještine i vještine upravljanja vremenom ključne su za uspjeh u svakoj karijeri. Izvannastavne aktivnosti također mogu pomoći u razvoju liderskih vještina ili vještina timskog rada [2].

Fakultet predstavlja veliku promjenu jer je to uglavnom za većinu studenata prvi put da žive daleko od doma i sami donose odluke. To podrazumijeva veliku prilagodbu, ali ujedno i uzbudljivo vrijeme za stjecanje neovisnosti. Kako teče proces rasta tako se i osoba mijenja. U svemu tome postoje trenutci kada se pojavi osjećaj izgubljenosti ili nesigurnosti u vezi budućnosti (pokazatelj da osoba brine o sebi) [2].

I za kraj jedna od zanimljivijih prednosti je prilika za putovanje i upoznavanje drugih drugih kultura. Mnogi fakulteti nude programe studiranja u inozemstvu koji studentima omogućuju da provedu semestar (ili dulje) živeći u drugoj zemlji. To se može smatrati vrijednim iskustvom jer omogućuje izravan uvid u nove običaje i perspektive [2].

2.2. Erasmus+ studentska mobilnost

Erasmus+ je program Europske unije za potporu obrazovanju, osposobljavanju, mladima i sportu u Europi. Program za razdoblje 2021. do 2027. godine snažno se fokusira na socijalnu uključenost, zelenu i digitalnu tranziciju te promicanje sudjelovanja mladih u demokratskom životu. Erasmus+ program nudi mogućnosti i prilike u područjima kao što su: visoko obrazovanje, strukovno obrazovanje i osposobljavanje, školsko obrazovanje, obrazovanje odraslih, mladi i sport. Na stranicama erasmus-plus.ec.europa.eu postoje izvješća o državama s interaktivnim podacima o razmjenama i projektima suradnje za sljedeće države članice: Austrija, Belgija, Bugarska, Hrvatska, Cipar, Češka, Danska, Estonija, Finska, Francuska, Grčka, Mađarska, Island, Irska, Italija, Latvija, Lihtenštajn, Litva, Luksemburg, Malta, Sjeverna Makedonija, Nizozemska, Njemačka, Norveška, Poljska, Portugal, Rumunjska, Srbija, Slovačka, Slovenija, Španjolska, Švedska i Turska [3].

U nastavku su detaljnije razmotreni podaci o Erasmus+ programu za Hrvatsku u 2023. godini. Hrvatska je jedna od 27 zemalja EU-a, čime ima pristup svim Erasmus+ aktivnostima kao što su razmjene, osposobljavanja, financiranja i slično. U nastavku za 2023. godinu su doneseni sljedeći zaključci [4]:

- U 2023. godini najveći broj projekata mobilnosti bio je u području školskog obrazovanja a slijede ga strukovno učenje i obrazovanje,
- Najveći udio sredstava u projektima mobilnosti potrošen je u sektoru visokog obrazovanja,
- Broj sudionika koji putuju u Hrvatsku povećava se svake godine,
- Tijekom 2023. godine došlo je do smanjenja broja sudionika koji su odlazili na razmjene u inozemstvo,
- Digitalna transformacija istaknuta je kao glavni prioritet u ukupnim bespovratnim sredstvima u 2023. godini.

Boravak u inozemstvu predstavlja izazovnu stvar koju poslodavci cijene i uzimaju u obzir prilikom intervjuiranja. Svaka osoba se može prijaviti na Erasmus+ program tako da se započetak obrati međunarodnom ili Erasmus+ uredu matičnog sveučilišta ili visokoškolske ustanove. Visokoškolska ustanova pošiljateljica dužna je odabrati kandidate na pošten i transparentan način. Maksimalno ukupno trajanje boravka u inozemstvu je 12 mjeseci unutar jednog studijskog ciklusa i shodno tome moguće je obaviti više od jedne razmjene unutar tog ograničenja. U nastavku se spominju dvije vrste mobilnosti [5]:

- Dugoročna mobilnost - minimalno 2 mjeseca do maksimalno 12 mjeseci u inozemstvu,
- Kratkotrajna mobilnost - između 5 i 30 dana u inozemstvu.

2.2.1. Izazovi i strahovi

Erasmus+ predstavlja uzbudljivo iskustvo koje je istovremeno veliki skok u nepoznato. Takvo iskustvo zahtijeva da osoba ostavi sve prethodno iza sebe i da se suoči s potpuno novim stvarima s kojima se prethodno nije imala prilike susresti. Na „Blogu Erasmus generacije“ navodi se nekoliko strahova koje Erasmus+ studenti mogu imati [6]:

- **Odlazak iz doma** - ako je napravljen popis za i protiv prije nego što se je predala prijava za Erasmus+, onda je odlazak iz svog doma bio poprilično visoko na lošijoj strani popisa. Taj loš osjećaj traje neko vrijeme ali nakon par provedenih dana kreće upuštanje u avanturu „ograničenu vremenom“. U nekim trenucima zna se pojaviti osjećaj kada nedostaje sve ono od čega se je otišlo (to je u situacijama kada je osoba sama) ili kada se razgovara s bližnjima (dobiva se osjećaj da se propušta nešto dok je osoba na drugom mjestu). „Biti daleko od onih s kojima ste nekada dijelili sve je pravi izazov, ali kada se vratite kući i otkrijte tko je prošao test „udaljenost“, shvatit ćete tko su vam pravi prijatelji“.
- **Financijska ne(odgovornost)** - pri početku Erasmus+ dobiva se veći dio stipendije koji je namijenjen za život tijekom cijelog Erasmus+. Bitno je da osoba ima vještinu upravljanja velikom količinom novca i da racionalno troši novce na potrebne stvari. Ovdje je posebno izazovno jer se radi o trošenju novca u stranoj zemlji pa se onda obično u početku potroše veće količine novca (npr. radi nepredvidivih troškova). Najbolja je stvar na početku donesti odluku što se želi raditi s novcem. Jesu li to putovanja, uživanje u hrani, tjedna kupovina, izlasci itd... Zanimljiva činjenica je da obično postoji želja za kombiniranjem/isprobavanjem svega.
- **Jezična barijera** - nedostatak povjerenja u znanje stranih jezika predstavlja jedan od glavnih razloga zašto studenti ne sudjeluju u programima kao što je Erasmus+. Pretjerana briga oko toga nema smisla jer Erasmus+ zahtijeva osnovno znanje stranog jezika i nije problem ako neko nije profesionalac u tome. Erasmus+ je upravo dobra prilika da se razbije ta barijera i nauči novi jezik.
- **Akademski život** - postoje studenti koji nikada prije nisu imali nastavu ili prezentiranje materijala na stranom jeziku. Bez obzira na to koliko osoba dobro zna jezik, učenje o svojoj budućoj profesiji na stranom jeziku može biti nezgodno. Upisuju se kolegiji o kojima se vrlo vjerojatno malo ili ništa ne zna, gdje predaju profesori koji nikada prije nisu viđeni. Bez obzira koliko god je to izazovno, iskušavanje drugačijeg načina podučavanja u kompletno drugačijem sustavnom obrazovanju je neprocjenjivo iskustvo.
- **Pakiranje** - je vrlo bitan faktor koji se općenito veže za svaku vrstu putovanja (trodnevni izlet, dvotjedni odmor) a u pravilu znači pakiranje cijelog života u jedan kofer. Pakiranjem se zapravo dobiva svijest o tome koliko stvari osoba posjeduje i koristi na dnevnoj bazi.

U nastavku su prikazani primjeri prijašnjih Erasmus+ studenta koji pričaju konkretno o prvom izazovu koji je naveden na prošloj stranici (Odlazak iz doma) [7]:

Ixan Curtamet – Rumunjska

Tijekom mog Erasmus+ imao sam osjećaj da mi nedostaje dom. Osjećao sam da propuštam puno toga u svojoj državi, falili su mi moji prijatelji. Imao sam periode kada baš nisam uživao na Erasmusu+ jer sam se osjećao loše zbog toga što sam bio tužan.

Katarina Bižanović – Hrvatska

U početku mi nije nedostajao dom. Pogodilo me nakon 3 ili 4 mjeseca kada su mi prijatelji počeli nedostajati, a to je obično bilo za vrijeme praznika, jer sam tada bila sama u stranoj državi.

Agnese Smioldo – Italija

Nedostajao mi je dom jer su mi nedostajale određene kulturološke stvari poput hrane. Falio mi je i moj tata.

Marc Dolcet Sadurni – Estonija

U početku mi je falio dom, ali uglavnom se puno toga događalo u isto vrijeme i nisam imao vremena razmišljati o tome. Uživao sam puno i upoznao sam mnogo ljudi.

Vitor Bizarro – Portugal

Mislim da je nemoguće da ti fali dom kad si na Erasmusu+. Kuhao sam kao kod kuće, družio se kao kod kuće i stalno sam zvao prijatelje iz svoje države i pritom uživao.

Minerva Martinez – Italija

Nije mi nedostajao dom jer su Italija i Španjolska jako slične. Ipak, nakon tri mjeseca znala sam ponekad osjetiti neki loš osjećaj.

Agnes Mfourmou – Belgija

Nije mi nedostajao dom jer sam stalno bila u kontaktu s prijateljima, dopisivali smo se, zvala sam mamu, slala im slike i videe. Morate uživati koliko god možete, jer će vam kasnije biti žao što odlazite s Erasmus+ i tada će vam nedostajati Erasmus+ dom. To je zapravo samo loš osjećaj jer se ne nalazite na nekom mjestu u nekom trenutku.

Kroz sljedeće poglavlje predstavljeno je koje su trenutno aktualne tehnologije i alati koji pomažu u današnjem informiranju i snalaženju među velikim brojem informacija.

3. Aktualne tehnologije i alati za uspješno snalaženje i informiranje

Unutar ovog poglavlja opisana je važnost inteligentnih pomoćnih sustava te je napravljena podjela na asistente, copilote, agente i chatbotove. Svaki od njih je kratko opisan i navedeno je nekoliko primjera. Za kraj ovog poglavlja opisano je korištenje inteligentnih asistenata u obrazovnom sustavu, njihova primjena i koji su izazovi takvog korištenja.

3.1. Važnost inteligentnih pomoćnih sustava

Davne 1961. godine IBM je predstavio prvi sustav koji je mogao prepoznati govor (izgovorene znamenke). Do 1990-ih kreirani su prvi komercijalni osobni asistenti koji su se aktivirali na postojanje glasa. Godine 2010. svijet je prvi put čuo za Sirija, Appleovog osobnog asistenta koji je pokrenut na temelju umjetne inteligencije. U to vrijeme Siri je predstavljao revolucionarno rješenje. Petnaest godina kasnije, korisnici su zatrpani s ogromnom količinom inteligentnih asistenata. Korisničko iskustvo (eng. User experience) je sada više nego ikada važno. Održavanje zadovoljstva korisnika/kupca i upravljanje njihovim očekivanjima zahtijeva stalne inovacije i poboljšanja. Korisnici žele da stranice budu intuitivne i učinkovite te da na neki način zadovolje njihove potrebe. AI asistenti su postali popularni alati koji pomažu u poboljšanju korisničkog iskustva. Točnije rečeno dodaju novi način interakcije između korisnika i pružatelja usluge i samim time povećavaju angažman, zadovoljstvo ili profit [8].

U svijetu umjetne inteligencije često se pojavljuju pojmovi poput bota, asistenta, copilota i agenta. Iako na prvi pogled djeluju kao sinonimi, svaki od njih ima svoju specifičnu svrhu, primjenu i skup prednosti.

Tip	Ključne značajke	Primjena	Primjer
Asistent	Personalizacija i multitasking	Dnevna produktivnost	Siri
Copilot	Specijalizirana podrška	Razvoj, IT	GitHub Copilot
Agent	Autonomija i proaktivnost	Upravljanje prijevarama, korisničko iskustvo	OpenAI Codex
Chatbot	Jednostavni ponavljajući zadaci	Osnovna korisnička podrška	Intercom

Tablica 1: Razlika između pojmova [9]

U okviru praktičnog rada napravljen je asistent koji omogućuje generiranje personaliziranih odgovora i pristupa specifičnim dokumentima čime pomaže korisnicima u pronalasku pravovaljanih i točnih informacija.

3.1.1. Asistent

AI asistenti donose inteligenciju automatizaciji razumijevanjem konteksta i time se personalizira interakcija kako bi se mogao obaviti niz zadataka. Ovi sustavi nadilaze skriptirane odgovore koristeći tehnologije poput obrade prirodnog jezika (eng. Natural processing language) ili strojnog učenja (eng. Machine learning). Kontekstualno su svjesni i razumiju korisnički unos što omogućava vođenje višestrukih razgovora. Integriraju se s alatima poput kalendara, aplikacijama za razmjenu poruku i poslovnim sustavima. Prilagođava se ponašanju korisniku kako bi se s vremenom poboljšao. Npr. mogu se koristiti za zakazivanje sastanaka, upravljanje e-poštom ili prilagođavanje rasvjete i temperature u pametnim domovima [9].

U nastavku su navedene prednosti intelligentnih asistenata [8]:

- **Brzina i jednostavnost** - AI asistenti omogućuju korisnicima da brzo i jednostavno dobiju ono što im je potrebno, smanjujući gubitak vremena. TikTok je dobar primjer kako usluga može neindirektno zadržati korisnika na njihovoj platformi jer su razvili mehanizam koji je naučio pretpostavljati što korisnik želi gledati,
- **Pružanje podrške 24/7** - AI asistenti su dostupni stalno, čime je omogućena podrška korisnicima kad god je to potrebno,
- **Prihvatanje od strane ljudi** - razgovor s AI asistentima se odvija kroz korisnička sučelja, gdje korisnici mogu komunicirati na ljudski razumljiv način s asistentima.

3.1.1.1. Siri

Siri je Appleov virtualni asistent za iOS, macOS, tvOS, i watchOS uređaje koji se koristi preko glasa i pokreće ga umjetna inteligencija. Siri odgovara na izgovorena pitanja korisnika tako što im odgovara putem zvučnika uređaja i prikazuje paralelno relevantne informacije na početnom zaslonu iz određenih aplikacija kao što je Kalendar. Siri se počeo izrađivati u neprofitnom istraživačkom institutu sa sjedištem u Kaliforniji nakon čega su pojedini članovi odlučili osnovati startup tvrtku pod nazivom Siri. Godine 2010. Apple je preuzeo Siri što je dovelo do brze integracije u iOS uređaje. Tijekom 2011. i 2012. godine Siri je bio na Iphone 4S i na iPad-ovima treće generacije. U lipnju 2024. Apple je na konferenciji WWDC 2024 najavio značajnu reviziju Siriya gdje je predstavljen Apple Intelligence koji predstavlja novi osobni AI sustav koji poboljšava Sirijeve mogućnosti. U početku glas Siriya je bio od umjetnice Susan Bennet. Zanimljiv slučaja je bio da sve dok joj nije prijateljica javila, Bennet nije bila svjesna da je postala Sirin glas. Iako Apple nikada nije priznao da je Bennet originalni Sirin glas, audio stručnjaci na CNN-u su to potvrdili. Karen Jacobsen je žena koja je posudila svoj originalni ženski glas za GPS sustave, dok je Jon Biggs, posudio svoj glas za mušku verziju Siriya. U međuvremenu Apple je razvio novi ženski glas za iOS11 pomoću tehnologije dubokog učenja snimanjem i kombiniranjem govora stotine kandidatkinja. Siri može obavljati razne zadatke kao što su: navigacija uputama (npr. Kakav je promet na putu do posla?), zakazivanje događaja i podsjetnika (npr. Slanje poruke za rođendan), pretraživanje weba (npr. Pronađi sliku psa) ili mijenjanje postavki (npr. Povećanje svjetline zaslona) [10].

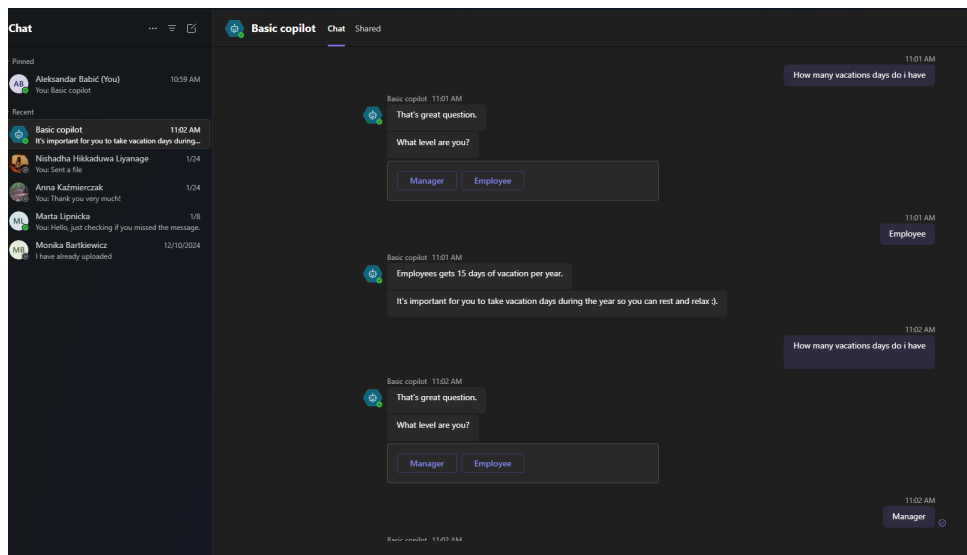
3.1.2. Copilot

Copilot je AI alat koji je u ulozi suradnika i primarno je osmišljen za poboljšanje ljudskih sposobnosti u složenijim zadacima. Za razliku od asistenata opće namjene, copiloti se usredotočuju na specifične domene, nudeći stručnost i proaktivnu podršku. Duboko su ugrađeni u specifične platforme čime pružaju podršku. Predviđaju korisničke potrebe analiziranjem konteksta i nude relevantne prijedloge ili rješenja. Na primjer, Github Copilot se koristi u razvoju softvera gdje predlaže isječke koda, otkriva pogreške i ubrzava razvoj automatizacijom rutinskih zadataka [9].

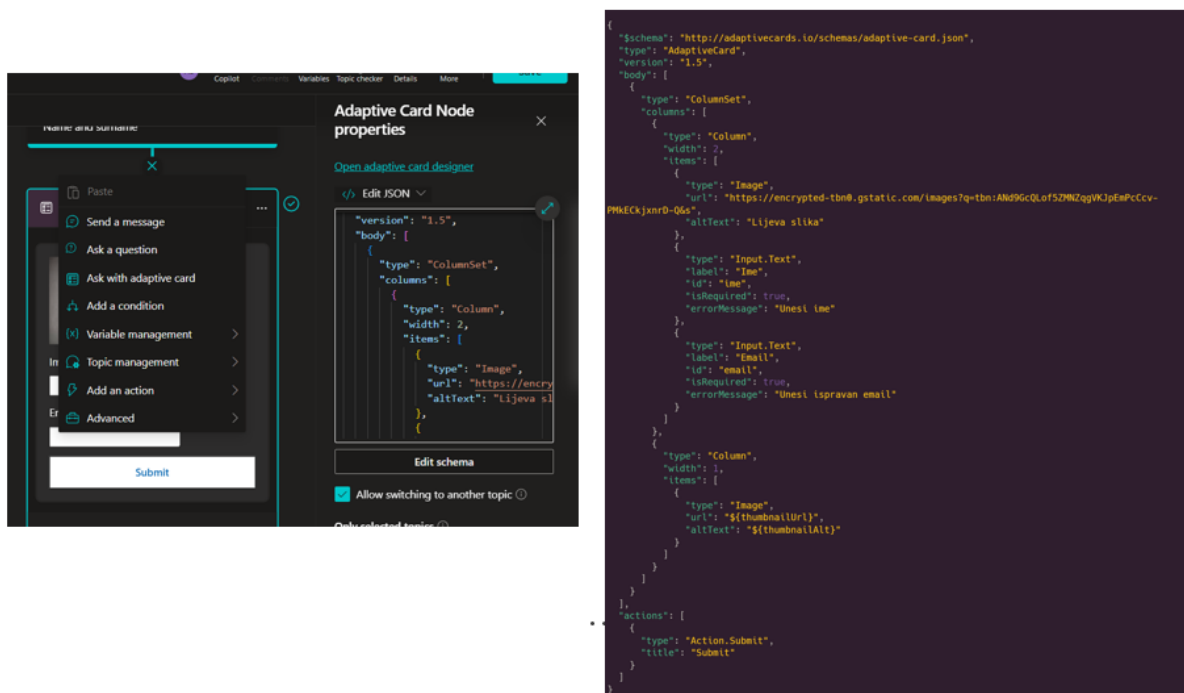
3.1.2.1. Microsoft Copilot Studio

Tijekom studentske prakse u jednoj varaždinskoj firmi bila je prilika raditi i istraživati alat Microsoft Copilot Studio. Rukovodstvo te tvrtke u tom trenutku je imalo mišljenje da korištenjem AI alata se znatno povećava produktivnost zaposlenika. Primarno bio je zadatak napraviti razna istraživanja u AI domeni jer su svi popularniji AI sustavi primjeri nekog oblika općenitog znanja. Ni jedan od njih ne zna ništa o poslovanju i domeni kojim se firma bavi. Unutar Copilot Studija mogu se razvijati copiloti u područjima kao što su: prodaja i podrška, informacije (podaci o otvaranju trgovina), zdravlje zaposlenika i godišnji odmori. Također, unutar sustava postoji nekolicina kreiranih predložaka s copilotima koji odgovaraju na karakteristična pitanja poput IT Helpdeska ili Team Navigatora. Primarno taj sustav je razvijen od strane Microsofta i koristi se za izradu copilota koji mogu odgovarati na specifična pitanja koja su usko vezana za neku domenu. U alatu se je isprobalo korištenje varijabli, entiteta, petlji, tijeka rada (eng. Workflow) ali ono što je bilo najzanimljivije je baza znanja (eng. Knowledge base) gdje se može spremati velika količina podataka koja je baza za odgovore. Osim toga kada se kreira agent on se može ugraditi u Microsoft Teams ili u web stranicu. Microsoft Copilot studio se bazira na kreiranju tema (eng. Topic) i dok god ima definirane teme za određene slučajeve od tamo će crpiti inspiraciju za odgovore. Postoje prilagođene (eng. Custom) i sistemske (eng. System). Prilagođene teme se koriste za komuniciranje s korisnikom (Dobar dan, doviđenja) dok se sistemske koriste za pozadinske operacije (inicijalna poruka pri početku konverzacije ili završetak razgovora). Kada copilot nema definiranu temu za odgovor na neko pitanje onda odlazi u sistemske teme i tamo otvara conversational boosting temu gdje je to zapravo tijekom rada koji traži informacije iz baze znanja [11]. Kao i svaki sustav, Copilot Studio ima svoje limitacije a to su: max 8000 zahtjeva po minuti u Dataverse okruženju, max datoteka veličine 512 MB, max 500 datoteka, max 200 fraza po temi i max 50 copilota po timu [12].

Osim toga isproban je i Bing Custom Search koji je vanjski servis koji se može koristiti za navođenje više izvora koji se koriste za odgovaranje na upite. Cilj je da se ne koristi cijeli internet nego da se koristi taj skup podataka koji je naveden. Putem "custom configuration ID-a" može se povezati Custom Bing Search i Copilot Studio. Naknadno isprobana je funkcionalnost Adaptive Cards Designer koji omogućava kreiranje vlastitih UI (eng. User Interface) elemenata koji se mogu koristiti unutar razgovora s korisnikom. Na sljedećoj slici može se vidjeti da je to JSON koji je potrebno importirati u Copilot Studio. Na stranici <https://adaptivecards.io/designer/> postoji veliki broj primjera koji su već kreirani i besplatni su za korištenje [11].



Slika 1: Primjer copilota kreiranog u Microsoft Copilot Studiu



Slika 2: Adaptive cards

Od drugih stvari koje još postoje u Copilot Studiju:

- Dvije vrste kreiranje entiteta (Copilot sadrži veliki broj entiteta koji se mogu koristiti ali se mogu kreirati i vlastiti (Closed List i Regular Expression)),
- Message variation koji omogućava postojanje nekoliko verzija incijalnih poruka koje se pojavljuju kada se copilot pokrene.

3.1.3. Agent

AI agenti predstavljaju vrhunac automatizacije i autonomije jer su sposobni samostalno donositi odluke i poduzimati radnje bez stalnog ljudskog vodstva. Dizajnirani su za složena okruženja a također se dinamički prilagođavaju kako bi postigli definirane ciljeve. Djeluju samostalno kako bi analizirali podatke i odredili prioritetne zadatke. Prate sustave i okruženja, predviđaju i rješavaju potencijalne probleme prije nego što eskaliraju. Pri radu s složenijim problemima, identificiraju optimalnija rješenja. Bankarski agenti zamrzavaju kompromitirane račune kada se otkrije neovlašteni pristup s visokorizične lokacije. Na primjer, prate stanje servera, otkrivaju anomalije, identificiraju kašnjenje pošiljaka i proaktivno obavještavaju kupca nudeći naknadu ili alternative i poduzimaju korektivne mjere bez ljudske intervencije [9].

3.1.3.1. ChatGPT Operator

ChatGPT Operator je agent koji može ići na web kako bi obavljao zadatke umjesto korisnika. Može pregledavati web stranicu i komunicirati s njom tipkanjem, klikanjem i pomicanjem. Pojavio se je u siječnju 2025. godine i trenutno je još u fazi razvoja gdje se nastoji poboljšati rješenje na temelju povratnih informacija korisnika. Od Operatora se može zatražiti da obavi širok raspon repetitivnih zadataka u pregledniku, kao što je ispunjavanje formi, naručivanje proizvoda, pa čak i stvaranje memeova. Operator je dostupan unutar PRO verzije na području SAD-a i pokreće se na temelju modela CUA (eng. Computer-Using Agent). CUA je obučan za interakciju s grafičkim korisničkim sučeljima (eng. Graphical User Interface) - gumbima, izbornicima i tekstualnim poljima koje ljudi vide na zaslonu. Ako se dogodi da se Operator susretne s izazovima ili ako napravi pogreške, u tom slučaju vraća kontrolu korisniku, osiguravajući glatko i korisničko iskustvo [13].

U nastavku se navodi jedan primjer koji ChatGPT Operator obavlja [14]:

1. Upravlja preglednikom i obavlja radnje u ime korisnika.
2. Analizira podatke na temelju YouTube veze.
3. Generira PNG sliku analize.
4. Ide na X.com, lokalno odabire sliku i prenosi je.
5. Lajka i komentira sliku.
6. Često traži potvrdu prije izvođenja radnji.
7. Može se koristiti za pretraživanje weba, kupnju ulaznica ili rezervaciju restorana.

OpenAI spominje kako planiraju u budućnosti dati na korištenje model CUA preko API-ja čime će omogućiti izradu vlastiti agenata svakome. Osim toga spominju kako planiraju proširiti Operator korisnicima Plusa, Teama i Enterprisea nakon što se uvjere u njegovu sigurnost i upotrebljivost u velikim razmjerima.

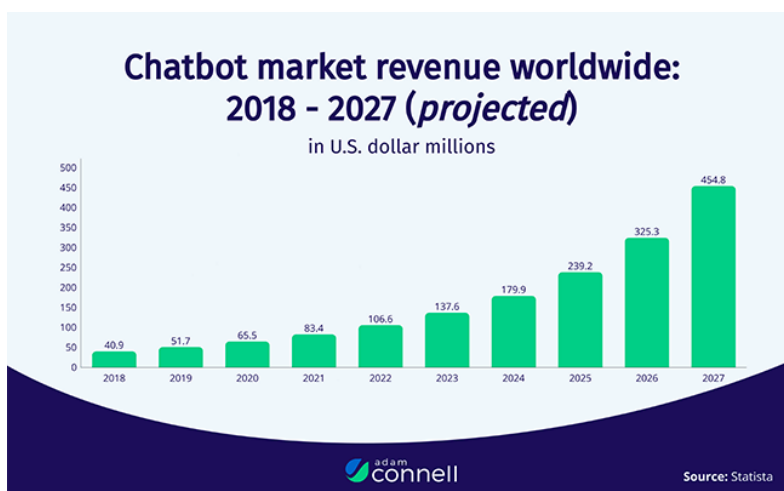
3.1.4. Chatbot

Chatbotovi su sustavi temeljeni na kreiranim pravilima koji su osmišljena za izvršavanje određenih zadataka putem unaprijed definirane interakcije. Iako posjeduju dosta ograničenja jako se dobro i brzo snalaze u dosljednom rješavanju jednostavnih repetitivnih zadataka. Djeluju na temelju stabla odlučivanja ili prepoznavanja ključnih riječi ograničavajući svoju interakciju na unaprijed postavljene scenarije. Koriste se za automatizaciju zadataka gdje se obrađuje veliki broj ponavljajućih upita poput rezervacija ili narudžbi. Njihova mana je da se ne mogu prilagoditi izvan svog programiranja što ih čini neprikladnim za složenije zadatke [9].

3.1.4.1. Intercom

Intercom je platforma za korisničku podršku koja olakšava komunikaciju između kupaca i tvrtki. Poboljšava interakciju s kupcima putem brojnih komunikacijskih kanala koji uključuju chat uživo i automatizirano slanje poruka. Intercom je moguće integrirati s drugim alatima i platformama kao što su Salesforce, HubSpot, Zendesk i Slack.

Chatbotovi su posebno zanimljivi jer su to inteligentni sustavi koji možda najviše rade s kupcima i samim time mogu dosta utjecati na profit i vrijednost tvrtke. Na sljedećoj slici može se vidjeti kako se pretpostavlja da će globalno tržište chatbotova do kraja 2027. godine vrijediti 455 milijuna dolara. Ova procijenjena brojka za 2027. godinu (455 milijuna USD) izračunata je na temelju tržišne vrijednosti u 2018. godini [15].



Slika 3: Globalno tržište chatbotova; Izvor

Također u sljedećem članku su doneseni sljedeći zaključci [15]:

- Tvrtke mogu smanjiti troškove korisničke podrške za 30 % korištenjem chatbotova,
- Trećinu korisnika chatbotova čine tzv. „napredni korisnici”,
- Najveći uočeni nedostatak chatbotova jest njihova nemogućnost odgovaranja na složena i personalizirana pitanja,
- Čak 55 % ljudi radije bi koristilo chatbota nego čekalo dostupnost agenta uživo.

3.2. Inteligentni asistenti u obrazovnom sustavu

Današnje obrazovanje se sve više oslanja na tehnologiju, a inteligentni asistenti postaju alat koji može poboljšati iskustvo učenja i poučavanja. Na primjer, svaki učenik može imati personaliziranu pomoć, a učitelji imaju više vremena za fokusiranje na ono što je zaista važno. Takvi asistenti mogu prilagođavati materijale, odgovarati na pitanja, ocjenjivati domaće zadaće čime se rasterećuje profesore. Jedna od najvećih prednosti koje donose inteligentni asistenti je personalizirano učenje. Umjesto univerzalnog pristupa, asistenti se mogu individualno posvetiti svakom učeniku/studentu.

3.2.1. Primjena inteligentnih asistenata u obrazovanju

Automatizacija procesa ocjenjivanja - ocjenjivanje pismenih zadataka, pružajući detaljne povratne informacije u roku od nekoliko minuta umjesto dana. Osim što skraćuje vrijeme ocjenjivanja profesorima, učenici dobivaju brže povratne informacije o svom radu. Jedan od primjera je Gradescope kojeg je razvio Turnitin. Kroz Gradescope moguće je ocjenjivati zadatke iz različitih predmeta, uključujući programiranje, fiziku, matematiku, kemiju, biologiju i ekonomiju. Također moguće je ocjenjivati programske projekte (u digitalnom obliku) i zadatke napisane na papiru. Neki od komentara profesora koji su koristili Gradescope su [16]:

- "S Gradescopom je čast ocjenjivati. Prije što sam radio u rasponu od 2 do 3 sata sada odradim u 15 minuta" - Romulo Chumacero - University of Chile
- "Statistika mi stvarno pomaže da shvatim što sljedeći put mogu drugačije objasniti kako bih pomogla svojim učenicima da bolje uče" - Katie Johnson - Florida Gulf Coast University
- "Gradescope omogućuje da svaki dan svojoj grupi od 60 učenika dam kratki kviz i da ih sve ocijenim tijekom 30-minutne vožnje vlakom kući." - Jesse Tov - Northwestern University

Smanjenje administrativnog opterećenja - na primjer, sustavi pokretani umjetnom inteligencijom mogu pratiti prisutnost, upravljati zadacima pa čak i predviđati kojim studentima je potrebna dodatna pomoć. Takav pristup pomaže u otkrivanju problema prije nego što postane još veći problem. Jedan od zanimljivijih citata od Rose Luckin, profesorice dizajna usmjerenog na učenike na University College London, je: "Prava moć inteligentnih agenata u obrazovanju ne leži u zamjeni učitelja, već u pojačavanju njihovog utjecaja. Obavljanjem rutinskih zadataka, AI asistenti oslobađaju edukatore da se usredotoče na ljudske elemente poučavanja, inspiraciju, mentorstvo i njegovanje vještina kritičkog razmišljanja" [17].

Prilagođavanje nastavnih materijala - analiziranje snaga, slabosti i načina učenja svakog učenika kako bi se stvorili prilagođeni materijali. Na primjer, ako postoji učenik koji se muči s razlomcima, asistent može pružiti dodatne resurse kako bi se poboljšalo njegovo razumijevanje [17].

Question 2 (points total):
 Pipeline the circuit below. Optimize throughput (inputs processed per ns) and cost (\$). You may add any number of the blocks in the circuit (A-F), edge-triggered flipflops, edge-triggered interleavers, an edge-triggered de-interleavers (see figure below). No other modifications are permitted. The latency of flipflops, interleavers, and de-interleavers is 0ns. The cost of any item (whether already there or one you add), regardless of which item is \$1. You have a total budget of \$25 and the components already drawn below cost \$6. Draw your answer on top of the diagram below if at all possible.

Cost = \$6 + \$18 = \$24
 Latency = 7.5 x 8 = 60ns
 Throughput = 1/7.5ns
 (If we add a flip at the output,)

Total Points
24.0 / 33.0 pts

1	+6.0	Correct interleaving concept
2	+6.0	Correct interleaving of multiple inputs
3	-3.0	Abused interleavers (too many outputs/inputs)
4	+6.0	Well-formed ("balanced" latch count)
5	+8.0	Correct extra stages for long-latency interleaved approach
6	-4.0	Not including latch for each input / extra latches if assuming interleavers latch

Slika 4: Ocjenjivanje zadatka - Gradescope [16]

3.2.2. Izazovi inteligentnih asistenata u obrazovanju

Jedan od najvećih izazova je pitanje privatnosti podataka. Obično sustavi zahtijevaju ogromne količine podataka o studentima da bi učinkovito funkcionirali, što ukazuje na opasnost od potencijalne zlouporabe tih podataka. Na primjer, studija Sveučilišta u Michiganu iz 2020. godine otkrila je da 86 % obrazovnih aplikacija umjetne inteligencije prikuplja osobne podatke od studenta, a samo 42 % ima jasne politike privatnosti [17].

Asistenti su dobri onoliko koliko su dobri podaci na kojima su obučeni. U obrazovnom sustavu, osiguranje kvalitete i relevantnosti tih podataka je od najveće važnosti. Kako bi rješenje bilo zadovoljavajuće škole i programeri moraju surađivati kako bi stvorili sustave koji su ne samo učinkoviti, već i etični i sigurni [17].

Implementacija inteligentnih asistenata zahtijeva značajnu tehnološku infrastrukturu. Mnoge škole, posebno u ruralnim područjima, nemaju potreban hardver, brzi internet ili softver za podršku ovim sustavima. Izvješće za obrazovanje iz 2022. godine kaže da gotovo 17 milijuna učenika u SAD-u još uvijek nema pristup internetu kod kuće [17].

Unatoč tim izazovima, integracija inteligentnih asistenata u obrazovni sustav donosi brojne prednosti. Sustavi pokretani umjetnom inteligencijom mogu automatizirati dugotrajne administrativne zadatke, oslobađajući učitelje da se usredotoče na ono što najbolje rade: inspiriranje i poučavanje učenika. Osim toga, AI asistenti mogu pružiti personalizirano učenje, povećanu produktivnost i učinkovitost, te poboljšano korisničko iskustvo [17].

Kroz sljedeće poglavlje je prikazano kako funkcioniraju inteligentni asistenti temeljeni na RAG (eng. Retrieval-Augmented generation) tehnologiji.

4. Retrieval Augmented Generation metoda

U ovom poglavlju obrađena je RAG (eng. Retrieval Augmented Generation) metoda, koja predstavlja suvremeni pristup u području obrade prirodnog jezika i kombinira sposobnosti generativnih modela s mehanizmima za dohvaćanje informacija iz vanjskih izvora. Poglavlje započinje pregledom povijesti a zatim slijedi teorijska osnova modela i njegova arhitektura kroz faze dohvaćanja i generiranja. Nadalje je opisana primjena RAG-a, prednosti i izazovi te je zaključeno s pregledom dostupnih alata i biblioteka za implementaciju RAG sustava.

4.1. Uvod u RAG

RAG je proces optimizacije izlaza velikog jezičnog modela pri čemu se referencira na autoritativnu bazu znanja izvan izvora podataka za obuku prije generiranje odgovora. Veliki jezični modeli (eng. Large Language Models - LLM) obučavaju se na ogromnim količinama podataka i koriste milijarde parametara za generiranje izlaza za zadatke poput odgovaranja pitanja, prevođenja jezika i dovršavanja rečenica. Ideja RAG-a je da proširi već moćne mogućnosti LLM-ova na određene specifične domene, a sve to bez potrebe za ponovnom obukom modela. Takav pristup poboljšava LLM-ove u smislu točnosti, korisnosti i relevantnosti [18].

Model velikog jezika može se zamisliti kao entuzijastičnog zaposlenika koji odbija biti informiran o aktualnim događajima, ali uvijek odgovara na pitanja s visokim samopouzdanjem. Nažalost, takav stav može negativno utjecati na povjerenje korisnika i nije nešto što bi trebalo postojati kod AI sustava. Iz takvih razloga RAG tehnologija preusmjerava LLM kako bi dohvatio relevantne informacije iz unaprijed definiranih izvora znanja [18].

Bez RAG-a, LLM uzima korisnički unos i stvara odgovor na temelju informacija na kojima je obučen. S RAG-om se uvodi komponenta za pronalaženje informacija koja koristi korisnički unos kako bi prvo privukla informacije iz novog izvora podataka. Nakon toga LLM kombinira novo znanje i svoje podatke za stvaranje boljih odgovora [18].

Primjerice, ako se postavi pitanje: "Koji planet u Sunčevom sustavu ima najviše satelita?", osoba može odgovoriti: "Odlično pitanje! Kao dijete sam volio astronomiju i pročitao sam članak o tome - mislim da je to Jupiter s 88 satelita." Međutim, takav odgovor ima nekoliko problema: osoba ne navodi konkretan izvor, iako nastupa s visokim samopouzdanjem ("ja to znam, pročitao sam") te informacija može bit zastarjela jer se temelji na sjećanju iz prošlosti [19].

Kada bi osoba rekla: "Idem prvo na NASA-inu stranicu i tamo ću potražiti tu informaciju", tada bi traženje odgovora bilo temeljeno na provjerenom i ažurnom izvoru. Nasuprot tome, LLM-ovi, poput GPT-a, često odgovaraju vrlo samouvjereno, primjerice: "Na temelju podataka na kojima sam treniran, odgovor je Jupiter." Iako zvuči uvjerljivo, takav odgovor može biti netočan, a korisnik često nije svjestan ograničenja modela ni datuma do kojeg je model treniran. U ovom slučaju, točan odgovor zapravo može biti Saturn, koji trenutno ima 114 poznatih prirodnih satelita. Rješenje za ovaj problem je korištenje spremišta znanja (eng. content

store) koje može sadržavati izvore poput internetskih stranica, PDF dokumenta ili baza podataka. LLM tada prvo pristupa tom vanjskom spremištu informacija, pretražuje ga, identificira relevantne sadržaje i tek tada generira odgovor na temelju tih aktualnih podataka. U RAG sustavu prompt se najčešće sastoji od tri dijela: instrukcija (što model treba učiniti), korisnikovog upita (pitanje ili zahtjeva) i informacija koje su dohvaćene iz spremišta znanja [19].

Još jedan primjer je situacija u kojoj Alice želi saznati koliko dana porodiljnog dopusta može dobiti. AI sustav koji ne koristi RAG odgovara veselo i netočno "Uzmi koliko želiš". Politike o porodiljnom dopustu su složene i razlikuju se ovisno o državi u kojoj se boravi. U ovom slučaju, veliki jezični model (LLM), umjesto da prizna ograničenje u znanju odgovarajući "Žao mi je, ne znam", generirao je frazu iz svog skupa za treniranje koristeći korisnički orijentiran, ali neutemeljen jezik [20].

4.1.1. Povijest i razvoj

Korijeni RAG-a mogu se pratiti od 1950-ih i 1960-ih kada su se koristili sustavi za pronalazanje informacija. Istraživači Hans Peter Luhn i Gerald Salton postavili su temelje za modele vektorskog prostora omogućujući računalima da pronađu dokumente koji su relevantni za korisničke upite. Obrada prirodnog jezika značajnije je napredovala u 1980-ima i 1990-ima putem statističkih jezičnih modela, poput n-gram modela [21].

Integracija pretraživanja i generiranja pojavila se u 2000-ima s razvojem sustava poput IBM Watsona, koji je koristio metodu pretraživanja za prikupljanje informacija i statističke modele za generiranje odgovora. Razvoj neuronskih mreža u 2010-ima dodatno je ubrzao ovu integraciju. Transformatori omogućuju značajan napredak u modeliranju jezika, dok tehnike gustog pretraživanja, poput pretraživanja temeljenog na ugrađivanju, dodatno poboljšavaju točnost pretraživanja [21].

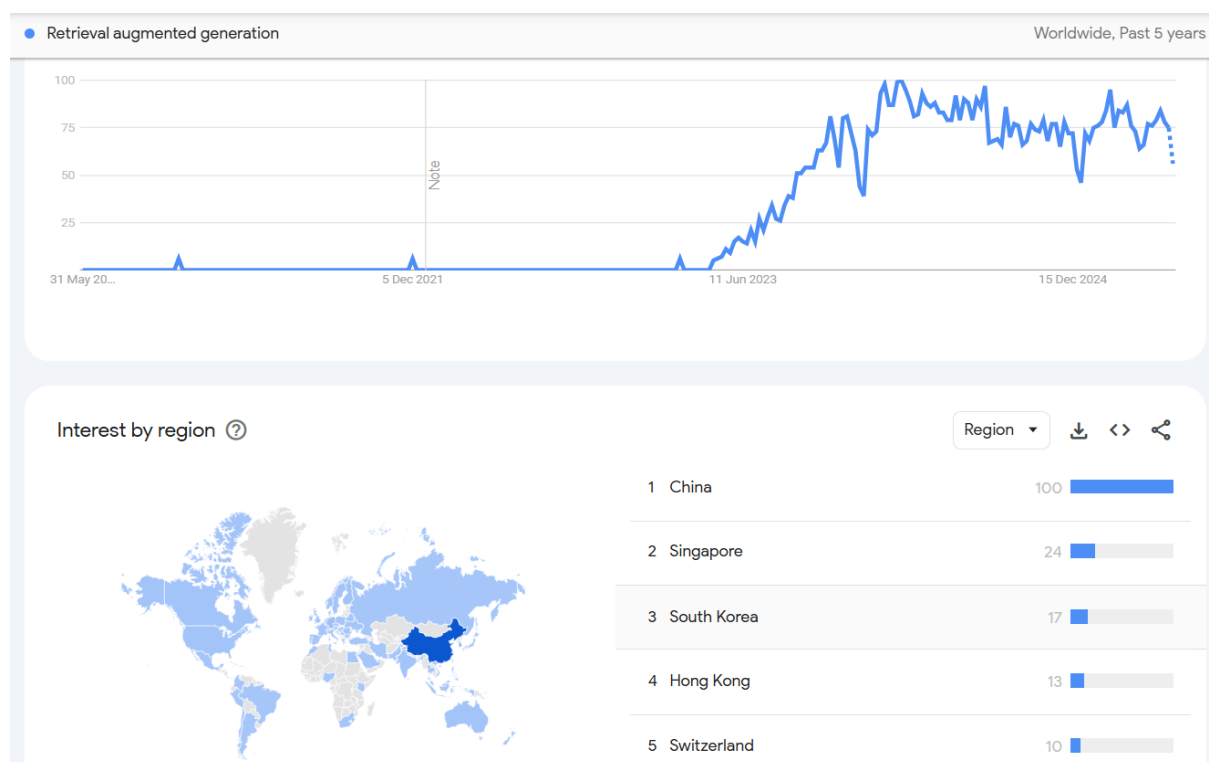
RAG se pojavljuje u radu "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" 2020. godine, gdje su autori predložili sustav u kojem vanjski model dohvaća relevantne dokumente kao odgovor na upit, a generativni model uvjetovan tim dohvaćenim znanjem daje odgovor. U radu su autori koristili bi-ekoder za pretraživanje, temeljen na pretraživanju gustih prolaza (eng. Dense Passage Retrieval) i T5 transformator kao generativni model, pokazujući izvanredne performanse na testovima koji zahtijevaju puno znanja. Ovaj pristup rješava ključne izazove s kojima se suočavaju modeli velikih jezika, poput halucinacija i zastarijelog znanja [21].

Objavljivanje RAG okvira potaknulo je nagli rast istraživanja, gdje je napravljeno nekoliko napredaka koja su se nadovezala na izvorni koncept [21]:

- **Poboljšani modeli pretraživanja** - modeli neuronskog pretraživanja poput ColBERTa postaju standard u RAG sustavima,
- **Skalabilnost i učinkovitost** - kako bi se obradile ogromne količine podataka. Tehnike poput pretraživanja najbližeg susjeda (ANN) i destilacije modela pomogle su u optimizaciji performansi,

- **Primjene specifične za domenu** - do 2023. godine RAG sustavi su se prilagođavali specifičnim slučajevima upotrebe, kao što su pravne i medicinske domene, gdje su točnost i objašnjivost ključne stavke. Dodatno se je istraživalo fino podešavanje kako bi se dobio što precizniji odgovor,
- **Multimodalni RAG** - proširivanje RAG-a kako bi uključio tekst, slike, zvuk i video. Proširivanje na područja poput obrazovanja i zabave,
- **Prilagođeno učenje i petlje povratnih informacija** - prilagodba prema korisnicima je ostvarena kroz razumijevanje njihovih povratnih informacija u stvarnom vremenu.

Kroz povijest RAG-a pokazuje se kako su se temeljna istraživanja u odvojenim područjima, poput pretraživanja i generiranja podataka, spojila kako bi se riješili izazovi u zadacima koji zahtijevaju puno znanja. Također može se vidjeti važnost interdisciplinirane suradnje u području umjetne inteligencije, gdje se otkrića često javljaju kombiniranjem različitih metodologija [21].



Slika 5: Popularnost RAG-a; Izvor

Važnost RAG-a povećala se posljednjih godina zbog pojave velikog broja aplikacija temeljenih na umjetnoj inteligenciji. Na slici 5. se može vidjeti kako uzlazna putanja popularnosti RAG-a kreće od petog mjeseca 2023. godine.

4.2. Arhitektura RAG modela

RAG se sastoji od faze pronalaženja i faze generiranja sadržaja. U fazi pronalaženja algoritmi traže relevantne informacije za korisnikov upit dok se u fazi generiranja generira tekst kako bi se odgovorilo na korisnički upit. U okruženju otvorene domene te činjenice uglavnom dolaze iz indeksiranih dokumenata na internetu. U okruženju zatvorene domene obično se koristi uži skup izvora radi dodatne sigurnosti i pouzdanosti. Prije LLM-a, agenti su kroz razgovor pratili ručni tijek dijaloga. Pretpostavljali su namjeru klijenta, dohvaćali tražene informacije i dostavljali odgovor za određeni scenarij. Za jednostavne upite, ova ručna metoda stabla odlučivanja funkcionirala je sasvim dobro. Problem takvog pristupa je da se je gubilo vrijeme na pisanje i predviđanje svakog pitanja koje bi korisnik mogao postaviti. U takvim slučajevima, ako tražena informacija nije definirana u dostupnim podacima, AI sustav neće biti u mogućnosti pružiti relevantan odgovor[20].

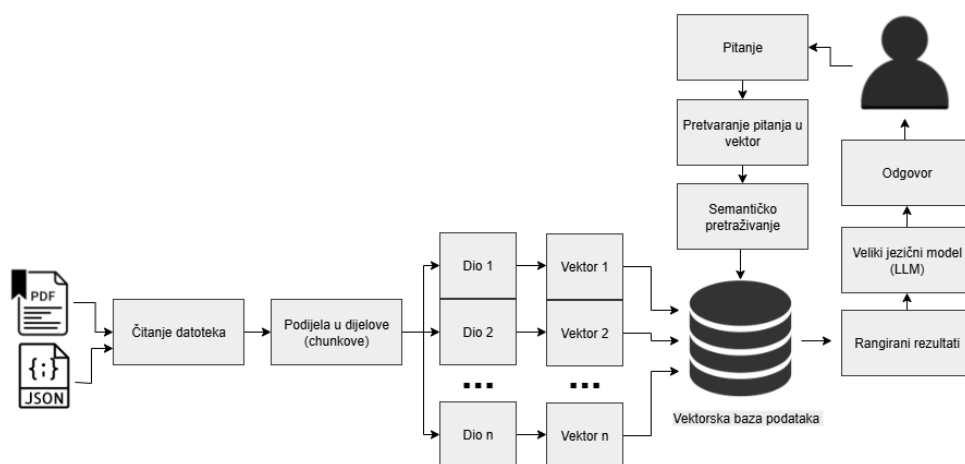
Generalno RAG arhitektura se može objasniti kroz sljedeće karakteristike [18]:

1. **Stvaranje vanjskih podataka** - novi podaci izvan originalnog skupa podataka koji su se koristili za treniranje LLM-a se zovu vanjski podaci (eng. external data). Oni dolaze iz više izvora podataka kao što su: API-ji ili baze podataka. Podaci mogu postojati u različitim formatima. Jedna od tehnika (eng. embedding language models) pretvara podatke u numerički oblik i sprema ih u vektorsku bazu podataka (kako bi generativni modeli umjetne inteligencije to mogli razumjeti).
2. **Preuzimanje relevantnih informacija** - Korisnički upit pretvara se u vektorski prikaz i uspoređuje se s vektorskim bazama podataka. Na primjer, ako korisnik postavi pitanje chatbotu "Koliko dana godišnjeg odmora imam?", AI sustav će morati preuzeti dokumente o politici godišnjeg odmora uz evidenciju prošlih odmora za pojedinog zaposlenika. Takvi dokumenti će biti vraćeni korisniku jer su relevantni za njegov upit.
3. **Proširivanje LLM upita** - Zatim, RAG model proširuje korisnički upit dodavanjem relevantnih dohvaćenih podataka u kontekstu. Prošireni upit omogućuje modelima velikih jezika generiranje točnih odgovora na korisničke upite.
4. **Ažuriranje vanjskih podataka** - Vrlo je bitno održati ažurne informacije i to se može ostvariti automatiziranjem procesa u stvarnom vremenu ili periodičnoj obradi.

Današnji AI sustavi bazirani na LLM-ovima mogu dati korisnicima odgovore bez da ljudi pišu definirane skripte. Samim nadovezivanjem RAG-a na LLM-ove smanjuje se potreba za ponovnim obučavanjem modela na novim primjerima [20]. Upiti korisnika nisu uvijek jednostavni, mogu biti dvosmisleno formulirani, složeni ili mogu zahtijevati znanje koje model nema. U takvim uvjetima LLM-ovi su skloni izmišljanju stvari. Rod Lastra opisuje velike jezične model kao "Zamislite model kao pretjerano nestrpljivog mlađeg zaposlenika koji izbrblja odgovor prije nego što provjeri činjenice" Lastra dodatno ističe važnost svjesnog upravljanja neizvjesnošću

u odgovaranjem, navodeći: “Iskustvo nas uči da zastanemo i kažemo kada nešto ne znamo. LLM-ovi moraju biti eksplicitno obučeni da prepoznaju pitanja na koja ne mogu odgovoriti” [20].

Sljedeća slika objašnjava kako RAG funkcioniše na konkretnom primjeru. Ulaz u RAG sustav mogu biti različite vrste dokumenta, poput PDF-ova ili CSV datoteka. Nakon toga čita se sadržaj dokumenta i dijeli se u manje dijelove (eng. chunks) koji se mogu lakše obraditi. Svaki od tih dijelova pretvara se u vektor koristeći model ugrađivanja (eng. embedding model) koji pretvara tekstualne podatke u numerički oblik. Ti vektori se zatim pohranjuju u vektorsku bazu podataka koja omogućuje brzi pretraživanje informacija. Nakon toga dolazi do korisničkog upita koji se također pretvara u vektor. Taj vektor se uspoređuje s vektorima u bazi podataka kako bi se pronašle relevantne informacije. Nadalje se rangiraju rezultati i odabiru oni najrelevantniji. Na kraju, LLM koristi te informacije za generiranje odgovora na korisnički upit.



Slika 6: Konceptualni tok korištenja RAG-a s LLM-ovima [18]

Jedna od zanimljivih stvari ovdje su vektorske baze podataka koje se mogu učinkovito indeksirati, pretraživati te je u njih jednostavno pohraniti veliku količinu informacija o čemu će se reći nešto detaljnije u jednom od sljedećih poglavlja.

4.2.1. Faza dohvaćanja (Retriever)

Retriever je komponenta koja je odgovorna za pronalaženje relevantnih dokumenata na temelju ulaznog upita. Retriever bilježi složenije semantičke odnose između upita i dokumenta, što dovodi do točnijih rezultata dohvaćanja. Retriever može ugraditi dokumente i upite u visokodimenzionalni vektorski prostor, gdje udaljenost između vektora odgovara relevantnosti dokumenta za upit. Dokumenti koji imaju veću "udaljenost" su manje relevantni za ulazni upit i time se mogu protumačiti kao nebitni. Retriever uzima ulazni upit, pretvara ga u vektor pomoću koder upita, a zatim pronalazi najslabije vektore dokumenta. Dokumenti povezani s tim vektorima se naknadno prosljeđuju generatoru. Broj dokumenta dohvaćenih u bilo kojem trenutku može se prilagoditi na temelju specifičnih zahtjeva zadatka i obično za zadatke koji su kompleksniji može se pozvati opsežniji skup dokumenata. Bitan aspekt kod retrievera je da se obrati pažnja na točnost dohvaćenih dokumenata tijekom pronalaženja. Postavljanje praga za udaljenost nije isključivo tehnička odluka, već i poslovna odluka. Prag bi trebao biti usklađen s općim ciljevima,

bilo da se radi o pružanju sveobuhvatnog pregleda ili navođenju konkretnih detalja [22].

Korisnik postavlja pitanje ili šalje upit nakon čega retriever traži relevantne dokumente u unaprijed definiranom izvoru znanja (npr. Wikipedia, interni podaci tvrtke ili online članci). Nakon toga se rangiraju dokumenti na temelju njihove relevantnosti za upit i za kraj se odabrani dokumenti proslijeđuju sljedećoj komponenti - generatoru - kako bi se pomoglo mu u generiranju odgovora. Retriever je važan jer za razliku od tradicionalnih AI modela, koji se oslanjaju samo na unaprijed obučene podatke, može dohvatiti informacije u stvarnom vremenu. Također korištenje vanjskih izvora pomaže u smanjenju AI halucinacija i omogućuje modelu da se referira na pozadinsko znanje, što dovodi do smislenijih odgovora [23].

4.2.1.1. BM25

BM25 (eng. Best Matching 25) je popularni algoritam koji koristi pojam frekvencijsko inverzne dokumentne frekvencije (eng. term frequency-inverse document frequency - TF-IDF) za rangiranje dokumenata na temelju relevantnosti. BM25 izračunava ocjenu relevantnosti dokumenta na temelju učestalosti pojavljivanja upita u dokumentu. Iako je učinkovit za podudaranja između ključnih riječi, BM25 može imati ograničenja u razumijevanju semantičkih odnosa. Unatoč tome BM25 se široko koristi zbog svoje jednostavnosti i učinkovitosti. Često se koristi za jednostavnije upite temeljene na ključnim riječima, gdje je važno brzo dohvaćanje relevantnih informacija [24].

4.2.1.2. Dense Passage Retrieval

DPR (eng. Dense Passage Retrieval) je predstavljen 2020. godine i predstavlja moderniji pristup pretraživanja informacija. Koristi gusti vektorski prostor u kojem su upiti i dokumenti kodirani u višedimenzionalne vektore. DPR koristi arhitekturu bi-ekodera, gdje su upit i dokument kodirani odvojeno, što omogućuje učinkovito pretraživanje najbližeg susjeda. Za razliku od BM25, DPR se ističe u pretraživanju semantičkih odnosa između upita i dokumenta, što ga čini pogodnim za složenije upite koji zahtijevaju dublje razumijevanje konteksta. DPR može pronaći dokumente koji su kontekstualno povezani s upitom, čak i ako ne sadrže točne ključne riječi [24].

4.2.1.3. REALM

REALM (eng. Retrieval-Augmented Language Model) je model koji integira pronalaženje informacija i generiranje teksta u jednom sustavu. Ključna inovacija REALM-a je da uči pronalaziti dokumente koji poboljšavaju performanse modela na određenim zadacima kao što je npr. sažimanje teksta. Tijekom treninga REALM ažurira retriever i generativni model zajedno, osiguravajući da je proces pronalaženja optimiziran za zadatak generiranja. REALM je treniran kako bi prepoznao dokumente koji nisu relevantni za upit, ali su korisni za generiranje odgovora. U literaturi se često spominje da REALM nadmašuje BM25 i DPR u zadacima koji zahtijevaju kombinaciju pronalaženja i generiranja informacija [24].

4.2.2. Faza generiranja (Generator)

Generator je komponenta koja koristi dohvaćene dokumente i izvorni upit kako bi generirala odgovor. To je obično veliki model transformatora kao što je GPT3.5, GPT4, Llama2, Falcpm, PaLM i BERT. Dohvaćeni dokumenti i ulazni upit ulančavaju se i unose u generator, koji zatim koristi kombinirani ulaz za generiranje odgovora pri čemu dohvaćeni dokumenti pružaju dodatni kontekst i informacije koje pomažu generatoru da bude informiraniji i precizniji u svojim odgovorima [22].

Generator je važan jer pretvara neobrađene podatke u koherentan i strukturiran tekst i osigurava da odgovori zvuče prirodno i privlačno. Također, spaja unaprijed obučeno znanje o umjetnoj inteligenciji s dohvaćenim informacijama u stvarnom vremenu za najbolji mogući odgovor [23].

4.2.2.1. T5

T5 (eng. Text-to-Text Transfer Transformer) je jedan od najčešće korištenih modela za zadatke generiranja u RAG sustavima. T5 je svestran model koji pretvara sve zadatke u tekstualni format, što ga čini pogodnim za širok spektar aplikacija. Može se koristiti za odgovaranje na pitanja, sažimanje ili generiranje dijaloga. Kroz praksu se pokazalo da RAG modeli temeljeni na T5 nadmašuju tradicionalne generativne model poput GPT-3 i BART-a u raznim zadacima kao što su odgovaranje na pitanja ili sažimanje teksta [24].

4.2.2.2. BART

BART (eng. Bidirectional and Auto-Regressive Transformers) je model koji je prikladan za zadatke koji uključuju generiranje teksta iz nečistih ili neuređenih podataka. Za enkoder BART koristi dvosmjerni pristup, što znači da može razumijeti kontekst cijelog ulaza, dok deko-der generira tekst na temelju tog konteksta. BART može rekonstruirati oštećene tekstove, što ga čini korisnim za zadatke poput izvlačenja informacija iz nepotpunih ili neurednih podataka [24].

4.2.2.3. GPT-4

GPT (eng. Generative Pre-trained Transformer) je veliki, multimodalni jezični model koji je sposoban prihvatiti tekstualne i slikovne ulaze te generirati tekstualne izlaze. Model je transformer arhitekture, treniran je da predviđa sljedeći token u tekstu, a naknadna prilagodba poboljšava željeno ponašanje. U usporedbi s prethodnikom GPT-3.5, postiže bolje rezultate na zajedničkim NLP testovima. Model je treniran korištenjem RLHF (eng. Reinforcement Learning from Human Feedback) metode kako bi se poboljšala sigurnost i preciznost njegovih odgovora te time smanjile halucinacije [25].

4.3. Evaluacija i primjena RAG metode

4.3.1. Usporedba: RAG vs. Fine-tuning

Fino podešavanje je metoda kod koje se uzima prethodno obučeni jezični model i dodatno se obučava na određenom, ciljanom skupu podataka kako bi se prilagodile njegove težine za određenu domenu. Tijekom finog podešavanja, model uči iz novih primjera kako bi mogao proizvesti specijaliziranije i točnije rezultate za određenu domenu. Fino podešavanje kreće odabirom modela opće namjene koji već široko razumije jezik. Nadalje se priprema skup podataka koji je relevantan za određenu domenu (Npr. modeli opće namjene mogu imati npr. poteškoća s medicinskom terminologijom.) Kvaliteta i relevantnost skupa podataka izravno određuje koliko će dobro fino podešeni model funkcionirati. Kroz npr. nadzirano učenje, model ponovno uči na novim podacima, prilagođavajući svoje interne parametre kako bi se bolje uklopili uzorci u skupu podataka za fino podešavanje. Proces učenja više puta propušta te primjere kroz neuronsku mrežu, izračunavajući pogreške između predviđanja modela i željenih izlaza, a zatim prilagođavajući težine kako bi se smanjile pogreške [26].



Slika 7: Fino podešavanje proces (generalno); Izvor

Kada se uspoređuju pristupi RAG i finog podešavanja, postoji nekoliko ključnih čimbenika koji mogu pomoći pri odabiru metode koja je prikladnija za konkretni problem.

Izbor između RAG-a i finog podešavanja je više tehnička odluka. RAG se ističe kada su potrebne ažurne informacije ili kada su ograničeni resursi za obuku. Fino podešavanje je izvrsno za specijalizirane zadatke koji zahtijevaju stručnost u određenoj domeni. Postoji opcija hibridnog pristupa gdje je moguće kombinirati ove dvije metode. Krajnja odluka bi trebala biti usklađena s potrebama i ograničenjima. Treba se uzeti u obzir proračun, učestalost promjena podataka, zahtjeve za privatnost, dostupnost inženjerskih resursa i razinu kontrole koja je potrebna nad rezultatima modela. Uspjeh bilo kojeg pristupa ovisi ponajviše o kvaliteti podataka tijekom cijelog procesa. Loša kvaliteta podataka potkopava RAG i fino podešavanje. Nebitni ili zastarijeli dokumenti smanjuju performanse RAG-a, dok krivo označeni ili pristrani podaci za obuku stvaraju nepuzadane fino podešene modele. Ovakvi problemi s kvalitetom podataka narušavaju performanse umjetne inteligencije, što dovodi krajnje štete i narušavanja povjerenja korisnika [26].

Kriterij	RAG	Fino podešavanje
Razmatranje troškova	RAG obično zahtijeva niža početna ulaganja, kasniji troškovi za hosting baze i infrastrukturu za pretraživanje.	Fino podešavanje zahtijeva značajne početne računalne resurse za obuku, ali rezultira nižim troškovima po upitu.
Privatnost podataka	RAG drži podatke odvojeno u kontroliranim bazama podataka, što omogućuje brisanje podataka kada je to potrebno.	Podaci su ugrađeni direktno u model, što neke organizacije smatraju zabrinjavajućom aktivnošću.
Inženjerski resursi	Stručnost u području baza podataka i pretraživanja.	Strojno učenje, pristup GPU-u i iskustvo s obučavanjem.
Željena razina kontrole modela	Manje izravne kontrole, ali lakša ažuriranja i ispravke.	Potpuna kontrola nad ponašanjem modela i stilom izlaza.

Tablica 2: Usporedba RAG pristupa i finog podešavanja modela [26]

4.3.2. Prednosti RAG sustava

U nastavku, kroz tablicu su prikazane ključne prednosti RAG sustava, koje objašnjavaju zašto je ova tehnologija sve češći odabir u izradi modernih AI rješenja. Navedene karakteristike posebno dolaze do izražaja u izradi inteligentnih asistentata koji trebaju pružati točne, relevantne i ažurirane informacije.

Tablica 3: Prednosti RAG-a [18]

Prednost	Opis
Isplativa implementacija	Razvoj AI sustava započinje korištenjem temeljnog modela (eng. Foundation models). Temeljni modeli su obično dostupni putem API ključeva i obučeni su na širokom spektru generaliziranih i neoznačenih podataka. Računalni i financijski troškovi za treniranje takvih modela su jako visoki.
Trenutne informacije	Iako su izvorni podaci LLM-a relevantni za specifične potrebe, teško je održati relevantnost. RAG se može koristiti za izravno povezivanje LLM-a s feedovima društvenih medija, web stranicama s vijestima i samim time pružiti korisnicima najnovije informacije.
Povećano povjerenje korisnika	RAG omogućuje prikazivanje točnih informacija s navođenjem izvora ili citata. Korisnici također mogu pretraživati izvorne dokumente ako im je potrebno dodatno pojašnjenje.
Više kontrole za razvojne programere	Učinkovitije testiranje i poboljšavanje aplikacija. Kontroliranje i mijenjanje izvora informacija. Rješavanje i ispravljanje problema.

4.3.3. Tipični slučajevi upotrebe

Kroz sljedeću sekciju istražena je primjena RAG-a. Na početku istaknut je jedan primjer koji je podijelio Siddharth Asthana u jednom od svojih članaka na LinkedInu. U članku se govori o tome kako je velika bolnička mreža integrirala RAG u svoj sustav podrške kako bi donijela bolje odluke. Sustav se povezao s elektroničkim zdravstvenim kartonima i višestrukim medicinskim bazama podataka, što je dovelo do [27]:

- Smanjenja pogrešnih dijagnoza za složene slučajeve za 30 %,
- Smanjenje vremena koje su liječnici proveli pregledavajući literaturu za 25 %,
- Povećanje ranog otkrivanja rijetkih bolesti za 40 %.

U nastavku je navedeno nekoliko domena u kojoj se RAG može primjenjivati [28]:

- **Sustavi za korisničku podršku** - kupac postavlja pitanje, chatbot dohvaća relevantne informacije iz izvora kao što su baze znanja, često postavljena pitanja ili zapisi kupaca i pritom koristi generativni model za izradu personaliziranog odgovora na temelju dohvaćenih podataka.
- **Sažimanje i pretraživanje dokumenta** - korištenje naprednih tehnika dohvaćanja informacija kako bi se poboljšale mogućnosti velikih LLM-ova. Na primjer, Google-ov Vertex AI Search koristi prvo algortime poput ANN kako bi brzo prikupio rezultate i zatim primjenjuje modele dubokog učenja za ponovo rangiranje kako bi se osiguralo da su najrelevantniji dokumenti prioritetni.
- **Medicinska dijagnostika i istraživanje** - korištenje ogromnih baza podataka medicinskog znanja, elektroničkih zdravstvenih kartona, kliničkih smjernica i medicinske literature kako bi se podržalo zdravstvene djelatnike u postavljanju točnih dijagnoza i dobro informiranih odluka o liječenju. Jedan od primjera je IBM Watson koji analizira podatke o pacijentima s opsežnom medicinskom literaturom i time pomaže liječnicima u dijagnosticiranju složenijih slučajeva. IBM Watson Health se također koristi za dijagnostiku raka i shodno tome predlaže preporuke liječenja.
- **Personalizirani sustavi učenja i podučavanja** - jedan od primjera je RAMO (eng. Retrieval Augmented Generation for MOOCs) koji se bavi problemom hladnog starta u preporukama tečajeva korištenjem LLM-ova za generiranje personaliziranih prijedloga tečajeva. RAMO pomaže u razumijevanju preferencija i ciljeva osobe, nudeći relevantnije mogućnosti tečaja. Na primjer, sveučilišta su počela uvoditi sustave podučavanja vođene RAG-om kako bi pomogli studentima u učinkovitijem snalaženju u materijalima, potičući dublje razumijevanje i bolje akademske rezultate.
- **Otkrivanje prijevara i procjena rizika** - sposobnost RAG-a da pristupi podacima u stvarnom vremenu i uključi ih tijekom postupka donošenja odluka. Financijske tvrtke poput JPMorgan Chasea koriste sustave za otkrivanje prijevara koje su vođene umjetnom inteligencijom. Takvi sustavi kontinuirano dohvaćaju i analiziraju

podatke u stvarnom vremenu iz različitih izvora kako bi se pratile transakcije i time otkrile potencijalne prijevare.

- **Preporuke proizvoda za e-trgovinu** - za razliku od tradiciionalnih sustava preporuka, RAG se dinamički prilagođava specifičnim potrebama kupca u stvarnom vremenu čime preporuke postaju relevantnije i točnije što rezultira povećanom prodajom.
- **Upravljanje znanjem u poduzeću** - posebno korisno kod velikih tvrtki s opsežnom dokumentacijom i izvorima podataka.

4.3.4. Izazovi RAG pristupa

LLM-ovi su ključna tehnologija umjetne inteligencije koja pokreće chatbotove. Priroda LLM-ova je takva da ponekad unose dozu nepredvidljivosti u generiranje odgovora. Osim toga, podaci o LLM obuci su statični i imaju krajnji rok za znanje koje posjeduju. Negativne strane LLM-ova uključuju [18]:

- Iznošenje lažnih informacija kada na njih nema odgovora (nepoželjni efekti nazvani - halucinacijama),
- Prikazivanje zastarjelih informacija kada korisnik očekuje aktualan odgovor,
- Izrada odgovora iz neautoriziranih izvora,
- Stvaranje netočnih odgovora zbog terminološke zbrke, različiti izvori obuke koriste istu terminologiju za opisivanje različite stvari.

Veliki jezični modeli mogu biti nekonzistentni što bi značilo da ponekad točno odgovore na pitanja, a drugi put nasumično pogađaju. Ako povremeno zvuče kao da nemaju pojma što govore, to je zato što ustvari nemaju pojma. LLM-ovi znaju kako se riječi statistički povezuju, ali ne znaju značenje riječi kao ni cjelinu njih, poznatu kao rečenica. RAG također smanjuje potrebu korisnika da kontinuirano treniraju model na novim podacima i ažuriraju njegove parametre. Na taj način, RAG može smanjiti računalne i financijske troškove pokretanja chatbotova [20].

Sljedeći izvor navodi sljedeće izazove kod RAG pristupa [29]:

- **Sadržaj koji nedostaje** - jedan od glavnih izazova je kada odgovor na upit korisnika nije prisutan u indeksiranim dokumentima. U takvim slučajevima sustav je sklon generiranju obmanjujućih odgovora ili u nekim slučajevima ne prepoznaje da mu nedostaju potrebne informacije. Na primjer, idealno ponašanje u takvim slučajevima je najbolje priznati poput: "Žao mi je, ne znam", iako to većina sustava ne uspijeva, što dovodi do smanjenog povjerenja korisnika. Ovaj izazov se može umanjiti povremenim dodavanjem podataka, implementacijom povratnih informacija kroz pogreške kako bi se ukazalo na nedostatak informacija i uvođenje prestanka generiranja odgovora kada odgovor ne zadovoljava potrebe korisnika.

- **Neoptimalno dohvaćanje i rangiranje** - u slučaju da odgovor postoji unutar indeksiranog skupa podataka, moglo bi se desiti da neće biti dovoljno visoko rangiran za dohvaćanje npr. zbog neoptimalnih algoritama rangiranja. Algoritmi rangiranja koji se oslanjaju isključivo na rezultate sličnosti često zanemaruju kontekst i specifičnost. Ovaj izazov se može umanjiti poboljšavanjem mehanizama rangiranja uključivanjem metapodataka kao što su vrste dokumenata, autorstvo ili datum objavljivanja. Također može se umanjiti s eksperimentiranjem s jačim modelima rangiranja kako bi se vidjelo hoće li se relevantniji dokumenti drugačije pozicionirati.
- **Proturiječne informacije** - proturiječne informacije mogu dovesti do toga da LLM generira zbunjujuće informacije (halucinacije). Na primjer, RAG sustav korisničke podrške dohvaća informacije zastarijele politike uz trenutne, uzrokujući da LLM generira netočne ili zbunjuće odgovore. To se može riješiti dodatnim filtriranjem nebitnih i kontradiktornih informacija prije nego što dođu do LLM-a. Dodatno trebalo bi još provesti testiranja i fino podesiti upite kako bi se usmjerila pozornost LLM-a na najrelevantnije / najnovije informacije.
- **Nepotpuni odgovori** - nepotpuni odgovori nastaju kada sustav ne uspije spojiti sve relevantne informacije. Na primjer, RAG sustav koji se bavi domenom prava i od njega se traži da sažme ključne točke tri različita slučaja, može se baviti samo jednim slučajem, izostavljajući kritične detalje iz ostala dva.
- **Izazovi s performansama i skalabilnošću u pronalaženju** - takvi izazovi nastaju kada se sustav bori s učinkovitim rukovanjem skupova podataka velikih razmjera. Kako veličina skupa raste, latencija dohvaćanja se povećava zbog računalnih troškova u pretraživanju, rangiranju i dohvaćanju relevantnih dokumenata.

4.3.4.1. Halucinacije

AI halucinacije su izlazi iz modela koji značajno odstupaju od stvarnosti i često predstavljaju netočne ili izmišljene informacije kao istinite. Halucinacije nastaju kao rezultat inherentnih ograničenja podataka za LLM obuku ili kada model ne uspije povezati namjeru ili kontekst upita s podacima potrebnim za generiranje smislenog odgovora. Iako je RAG osmišljen kako bi pomogao u smanjenju AI halucinacija, RAG halucinacije se i dalje mogu pojavljivati. Na takve stvari treba pripaziti jer na primjer, pretplatnik mobilne mreže može dobiti netočan odgovor o svom prosječnom mjesečnom računu od operaterovog RAG chatbota - jer podaci tvrtke možda uključuju račune koji nisu bili njegovi [30].

AI halucinacije se mogu suzbiti kroz nekoliko pristupa [30]:

- **Fino podešavanje provjerom činjenica** - postoji sloj koji projerava činjenice prije nego što se generira odgovor. Nedosljednosti se označavaju, što potiče LLM da poboljša svoj rezultat na temelju činjeničnih temelja.
- **Navođenje izvora** - razumijevanje načina na koji generativna umjetna inteligencija dolazi do odgovora ključno je. Korisnicima se pokazuju izvori koji su korišteni za

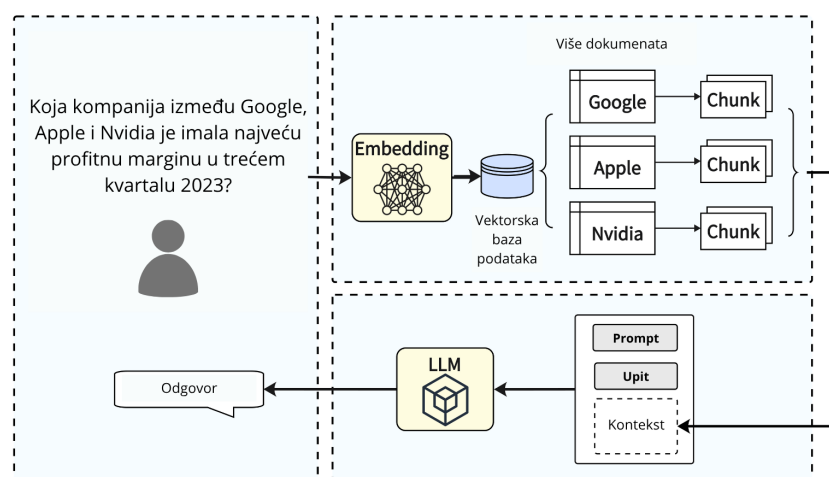
generiranje odgovora, čime je osigurana pouzdanost i transparentnost.

- **Povezivanje modela s javnim podacima više kvalitete** - daje se prednost visokokvalitetnim, raznolikim i činjeničnim informacijama. Tehnike poput čišćenja podataka i filtriranja pristranosti mogu pomoći u smanjenju halucinacija.
- **Korištenje RAG-a za proširenje LLM-ova s privatnim organizacijskim podacima** - nadopunjavanje znanja s privatnim podacima organizacije može smanjiti rizik od izmišljenih odgovora.

RAG ne može u potpunosti eliminirati halucinacije i ograničen je kvalitetom podataka, kontekstualnom svijesti (možda se neće svaki put najbolje razumijet što korisnik želi - propuštanje poante prompta), razmišljanjem (zdravi razum) i kreativnošću modela.

4.3.4.2. Upravljanje višestrukim izvorima

Upravljanje višestrukim izvorima dolazi kada model istovremeno koristi podatke iz različitih vanjskih baza kako bi generirao odgovor koji je što točniji, relevantniji i potpuniji. Prije svega, različiti izvori često donose različite informacije koje nisu uvijek usklađene. To može podrazumijevati kontradiktorne podatke, različite perspektive te redundantne podatke. Model se suočava s izazovom kako inteligentno integrirati te podatke bez stvaranja zbunjujućih ili netočnih odgovora. Ako model pogrešno spoji podatke iz nespojivih izvora podataka, povećava se rizik od generiranja lažnih informacija (halucinacija) ili gubitka relevantnih detalja. Jedan od pojmova koji se spominje kroz literaturu je višeskočno zaključivanje (eng. multi hop reasoning) kod kojeg se povezuju informacije iz više izvora kako bi se odgovorilo na složenija pitanja. Posebno se ističe u situacijama kada se problem ne može riješiti u jednom dohvat u podataka već zahtijeva kombiniranje više dohvaćanja iz više različitih izvora podataka. Takav pristup nudi iterativno upravljanje procesom traženja i generiranja čime se povećava efikasnost i točnost modela [31].



Slika 8: Višeskočno zaključivanje - primjer; Izvor

Glavna ideja primjera je da se ne oslanja samo na jedno pretraživanje ili jedan izvor, već da se kroz više iteracija i izvora spojene informacije upotrebe kako bi se dobio detaljniji odgovor.

4.4. Dostupni alati i biblioteke

U ovoj sekciji su navedeni neki od najpopularnijih alata i biblioteka koje se koriste za implementaciju RAG sustava. Izvršen je pregled i analiza svakog alata nakon čega je donesen zaključak koji od njih su korišteni u praktičnom dijelu rada. Na sljedećoj slici može se vidjeti kratki sažetak alata i biblioteka koje se mogu koristiti za implementaciju RAG sustava.



Slika 9: Sažetak dostupnih alata i biblioteka za razvoj RAG sustava; Izvor

4.4.1. Okviri (frameworkovi) za upravljanje RAG-om

U nastavku su navedeni neki od najpopularnijih okvira koji se koriste za jednostavniju izgradnju, testiranje i održavanje sustava temeljenih na RAG pristupu. Pružaju funkcionalnosti za povezivanje velikih jezičnih modela s vanjskim izvorima podataka kao što su baze podataka.

4.4.1.1. Haystack

Haystack je platforma za obradu prirodnog jezika otvorenog koda koja je specijalizirana za stvaranje RAG cjevovoda za sustave pretraživanja i odgovaranja na pitanja. Framework pruža komponente za dohvaćanje dokumenta, odgovaranje na pitanja i generiranje teksta, omogućujući korisnicima da brzo i jednostavno izrade RAG aplikacije. Haystack se također povezuje s modelima kao što su BERT i RoBERTa, što poboljšava njegovu sposobnost rukovanja s težim upitima. Također Haystack ima API koji je prilagođen korisniku i korisničkom sučelju temeljenom na webu, omogućujući korisnicima jednostavnu interakciju sa sustavom [32].

Značajke koje Haystack nudi su [32]:

- Podržavanje pozadinskih sustava za pohranu kao što su Elasticsearch, FAISS, SQL i InMemory,
- Pretraživanje na temelju ključnih riječi pomoću BM25,
- GenerativePipeline - kombiniranje retrivera i generatora (GPT - 3/4),
- TransformersReader - ekstrakcijska kontrola kvalitete koristeći Hugging Face modele,

- DensePassageRetriever - dohvaća gusto ugrađene vektore koristeći DPR (eng. Dense Passage Retrieval),
- EmbeddingRetriever - custom embeddings koristeći Hugging Face modele.

Vrlo lagano se može instalirati lokalno na Linux sustavu pomoću pip-a ili conde (pip install haystack-ai). Dokumentacija se nalazi na sljedećem linku - <https://docs.haystack.deepset.ai/docs/intro>.

4.4.1.2. LangChain

Langchain je Python paket otvorenog koda koji pruža sveobuhvatanu osnovu za izgradnju aplikacija s velikim jezičnim modelima. Kombinira modularni i fleksibilni dizajn s sučeljem visoke razine, što ga čini idealnim za razvoj RAG sustava. Također, LangChain omogućuje jednostavnu integraciju različitih izvora podataka, uključujući dokumente, baze podataka i API-je što pomaže u procesu generiranja odgovora. Langchain nudi širok raspon funkcionalnosti i omogućuje korisnicima izmjenu i kombiniranje različitih komponenti kako bi odgovarale jedinstvenim zahtjevima aplikacije, što olakšava stvaranje dinamičkih i prilagodljivih aplikacija [32].

Značajke koje LangChain nudi su [32]:

- Integracija s vektorskim bazama podataka kao što su Pinecone, Chroma i FAISS,
- Učitavanje i dohvaćanje podataka iz baza podataka, API-ja i lokalnih datoteka,
- Retrivi ukljućuju BM25, Chroma, FAISS i druge,
- Učitavanje PDF, teksta, web scrapinga i SQL/NoSQL baza podataka,
- Generiranje dinamićkih upita pomoću predloženih struktura.

Langchain se može instalirati lokalno na Linux sustavu koristeći pip (pip install -U langchain). Dokumentacija se nalazi na - <https://python.langchain.com/docs/introduction/>

4.4.1.3. LlamaIndex

LlamaIndex je robusna biblioteka otvorenog koda koja se usredotoćuje na učinkovito indeksiranje i dohvaćanje iz ogromnih skupova podataka. LlamaIndex koristi napredne tehnike kao što je pretraživanje vektorske sličnosti i hijerarhijsko indeksiranje kako bi se omogućilo brzo i točno dohvaćanje relevantnih informacija, poboljšavajući mogućnosti generativnih jezičnih modela. Lako se povezuje s uobičajenim velikim jezičnim modelima, omogućujući umetanje primljenih podataka u proces stvaranja i čineći ga učinkovitim alatom za poboljšanje performansi RAG sustava [32].

- Optimizirano je dohvaćanje s malom latencijom,
- Moguće je učitavanje razlićkih vrsta podataka kao što su: TXT, PDF, DOC, CSV, API-je, baze podataka i web scraping,

- Kombiniranje modela (OpenAI ili Hugging Face) s vektorskim bazama podataka,
- Sadrži više indeksa kao što su:
 - **Indeks pohrane vektora** - se koristi za pohranu podataka kao vektora, što omogućuje brzo pretraživanje sličnosti kao što su sustavi za dohvaćanje dokumenata i sustavi preporuka,
 - **Indeks popisa** - je jednostavan, sekvencijalan indeks za manje skupove podataka koji omogućuje brzo linearno pretraživanje,
 - **Indeks stabla** - koristi hijerarhijsku struktru za izvođenje učinkovitih semantičkih pretraživanja čime je idealan za složene upite kod podataka gdje je jasna hijerarhija,
 - **Indeks tablice ključnih riječi** - tablica mapiranja se koristi kako bi olakšala pretraživanje temeljeno na ključnim riječima.

LlamaIndex je moguće instalirati pomoću naredbi `pip install llama-index` ili `pip install llama-index-core`. Dokumentacija se nalazi na sljedećem linku <https://docs.llamaindex.ai/en/stable/>.

4.4.2. Vektorske baze podataka

Tradicionalne baze podataka pohranjuju podatke u strukturiranom formatu (redci i stupci), dok vektorske baze podataka su optimizirane za rukovanje visokodimenzionalnim vektorskim podacima i podržavaju operacije poput pretraživanja sličnosti i pretraživanja najbližih susjeda (k-NN). Umetanje podataka se vrši kroz generiranje ugrađivanja (eng. embeddings) iz neobrađenih vrsta podataka (npr. teksta) i pohranjivanje tih ugrađivanja u vektorsku bazu podataka. Vektorske baze podataka ključne su kako bi omogućile LLM-ovima da daju točnije i skalabilnije informacije. Osim toga, vektorske baze podataka omogućuju rad s bilo kojim drugim tipovima podataka, uključujući slike, videozapise i zvuk, čime se proširuju mogućnost RAG sustava. Većina podataka koja se generira svakodnevno su nestrukturirani podaci, bilo to tekst, slike, videozapisi ili slično. Nestrukturiranim podacima nedostaje strogi format, što otežava upravljanje konvencionalnim bazama podataka i iako se ponekad čine beznačajnim, oni sadrže vrijedne informacije koje imaju ogroman potencijal za umjetnu inteligenciju [33].

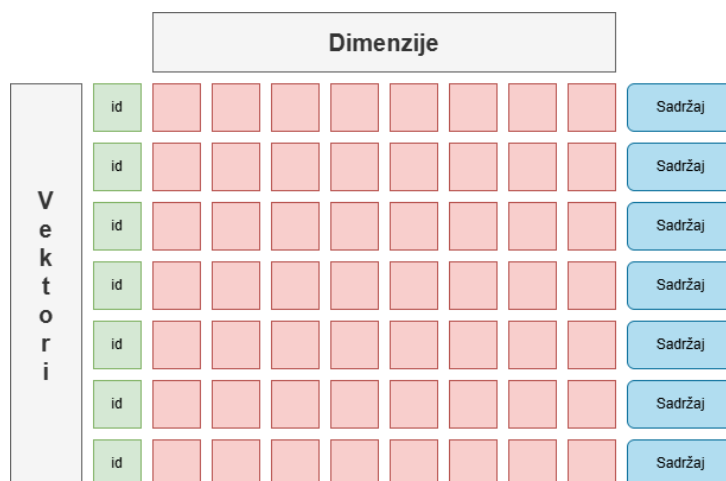
Tradicionalne baze podataka poput OLTP i OLAP odlično upravljaju s strukturiranim podacima, međutim kada se pojave podaci koji se ne mogu lako kategorizirati poput sadržaja unutar PDF datoteka stvari počinju biti komplicirane. Takva baza ne može razumijeti što se nalazi u dokumentu, ne može ga kategorizirati ili pretraživati. Vektorske baze podataka dobile su na značaju u posljednjih nekoliko godina zbog porasta strojnog učenja i dubokog učenja. Vektorska ugrađivanja pretvaraju složene podatke (npr. slike ili tekst) u visokodimenzionalne vektore tako da su slične stavke bliže jedna drugoj u vektorskom prostoru [33].

Tradicionalne baze podataka su neučinkovite pri pretraživanju sličnosti u visokodimenzionalnim prostorima. Takav problem vrlo dobro rješavaju vektorske baze podataka koje vrlo

dobro indeksiraju i pretražuju opsežne skupove podataka. Kako bi se proveo rad s vektorskim bazama podataka, potrebno je koristiti vektor upita koji obuhvaća kriterije pretraživanja. Sljedeći korak je korištenje metrike sličnosti za određivanje blizine između vektora upita i vektora u bazi podataka. To može uključivati metriku kao što su kosinusna sličnost, euklidska udaljenost ili Jaccardov indeks. U prošlosti su samo velike tehnološke tvrtke koje su imale resurse za njihovo stvaranje i održavanje koristile vektorske baze podataka. S obzirom na napredak tehnologije, pojavile su se mnoge vektorske baze podataka otvorenog koda koje su dostupne za korištenje i implementaciju u RAG sustavima. Sljedeći grafikon prikazuje kada se koriste vektorske baze podataka u odnosu na tradicionalne baze podataka [33].

Karakteristika	OLTP baza podataka	OLAP baza podataka	Vektorska baza podataka
Struktura podataka	Redci i stupci	Redci i stupci	Vektori
Podaci	Strukturirani	Strukturirani / nestrukturirani	Nestrukturirani
Metoda upita	SQL (Transakcije)	SQL (agregacije i analitički upiti)	Vektorsko pretraživanje na temelju sličnosti
Fokus	Optimizirano za ažuriranje	Optimizirano za čitanje	Kontekst i semantika
Slučajevi upotrebe	CRM, obrada narudžbi	Poslovna inteligencija, skladišta	RAG, otkrivanje anomalija

Tablica 4: Usporedba OLTP, OLAP i vektorskih baza podataka [33]



Slika 10: Vektorska baza - primjer; Izvor

Prethodna slika prikazuje strukturu vektorske baze podataka. ID (zelena polja) predstavlja jedinstveni identifikator svakog vektora, primjerice doc01. Svaki redak u crvenoj matrici odgovara jednom vektoru, odnosno nizu brojeva koji predstavlja semantičko značenje određenog teksta. Plava polja označena kao payload sadrže dodatne informacije povezane s vektorom, poput originalnog teksta, metapodataka kao što su jezik i izvor, te putanje do dokumenta. Dimenzije vektora, prikazane u gornjem dijelu slike, određuju širinu svakog vektora, odnosno broj numeričkih vrijednosti koje ga čine.

4.4.2.1. FAISS

FAISS (eng. Facebook AI Similarity Search) je robusna vektorska baza podataka koju je razvio Facebook-ov istraživački tim za umjetnu inteligenciju. Vrlo je učinkovita i brza, što je čini savršenim izborom za razvoj aplikacija u stvarnom vremenu specifičnih za industriju. Dostupno je nekoliko opcija indeksiranja, učinkovito grupira i izgrađena je na Pythonu i C++ jeziku. Jedne od poznatijih tvrtki koje koriste FAISS su Meta i Pinterest [34].

4.4.2.2. Qdrant

Qdrant je vektorska baza podataka koja je optimizirana za pretraživanje po sličnosti i omogućuje brzo i učinkovito pretraživanje velikih skupova podataka. Od dodatnih mogućnosti Qdrant nudi: mogućnost preciznog pretraživanja, pametno pohranjivanje vektora, skalabilni dizajn prilagođen oblaku. Klijenti Qdranta su: Bosch, Johnson i Johnson i Discord [34].

4.4.2.3. Pinecone

Pinecone je vektorska baza podataka u oblaku (eng. cloud-native) zatvorenog koda koja nudi upravljivost, skalabilnost i visoku učinkovitost. Jednostavnost, podrška za hibridne upite i uska integracija s strojim učenjem čine Pinecone popularnim izborom za RAG sustave. Značajke koje Pinecone nudi su: ažuriranje u stvarnom vremenu bez zastoja, visokodimenzionalno vektorsko pretraživanje u velikoj mjeri i integracija s Pythonom, Langchainom i drugima. Neki od klijenta Pinecone su: Microsoft, Notion, Hubspot i Shopify. Pinecone je vektorska baza podataka u oblaku koja je potpuno upravljana i ne zahtijeva nikako odražavanje infrastrukture [34].

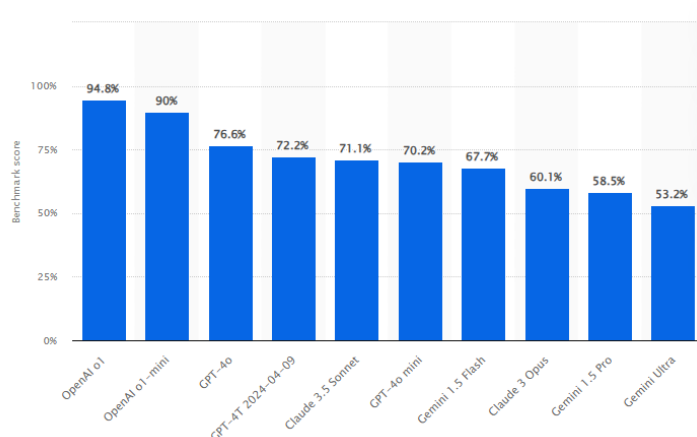
4.4.3. Veliki jezični modeli

Veliki jezični model je napredni AI model koji može razumijeti i generirati ljudski jezik. U srži LLM-ova je strojno učenje, koje je podskup umjetne inteligencije i omogućuje sustavima učenje iz podataka i poboljšanje tijekom vremena bez eksplicitnog programiranja. Također koristi se duboko učenje kako bi se analizirala ogromna količina podataka. Neuronske mreže su jedna od opcija kod kojih se obučava model na velikim skupovima podataka kako bi se prepoznali odnosi između riječi i fraza. LLM-ovi se mogu kategorizirati na mnogo načina no najčešći način kategorizacije je prema njihovoj arhitekturi i funkcionalnosti [35]:

- Generativni modeli - osmišljeni za stvaranje visokokvalitetnog teksta nalik ljudskom u raznim formatima (GPT-4),
- Modeli enkodera-dekodera - specijalizirani za zadatke poput strojnog prevođenja i sažimanje teksta (primjer T5),
- Modeli samo s enkoderom - fokusirani na razumijevanje značenja teksta (primjer BERT),

- Multimodalni LLM-ovi - obrađivanje i generiranje više vrsta podataka, uključujući tekst, slike i zvuk. Koriste se zadatke poput opisivanja slika, analize videa i multimodalnog pretraživanja kako bi se stvorila bogatija interakcija između različitih oblika podataka.

Na sljedećoj slici se može vidjeti kako je model OpenAI o1 predstavnik u rješavanju matematičkih problema za 2024. godinu.



Slika 11: Popularnost LLM-ova u rješavanju matematičkih problema za 2024 godinu; Izvor

4.4.3.1. OpenAI

OpenAI je jedan od začetnika i najpoznatijih pružatelja usluga u području umjetne inteligencije. Poznat je po razvoju GPT modela, uključujući GPT-3 i GPT-4, kao i Codexa za generiranje koda. Od dodatnih mogućnosti generira tekst, razumije jezik, pomaže pri kodiranju i može se koristiti za razvoj AI sustava i drugih aplikacija. Dostupan je putem OpenAI API-ja i putem Microsoftove usluge Azure OpenAI [36].

4.4.3.2. Hugging Face

Hugging Face je platforma koja nudi širok spektar modela otvorenog koda a poznat je po svojoj biblioteci Transformers i Model Hub-u. Hugging Face nudi različite modela poput BERT-a, GPT-2 i T5-a koji se mogu koristiti za različite zadatke. Transformers biblioteka koja omogućuje jednostavno korištenje i integraciju najsuvremenijih modela dubokog učenja. ModelHub je platforma na kojoj programeri i istraživači mogu prenositi, dijeliti i surađivati na modelima. Također Hugging Face potiče okruženje u kojem se dijeli znanje i resursi čime se ubrzava razvoj i potiče se na inovacije u području umjetne inteligencije [36].

Zaključak je da su svi navedeni alati i biblioteke korisni za implemetaciju RAG sustava, ali za potrebe ovog rada korištena je kombinacija Langchaina, llama modela s Groq platforme i FAISS vektorske baze podataka.

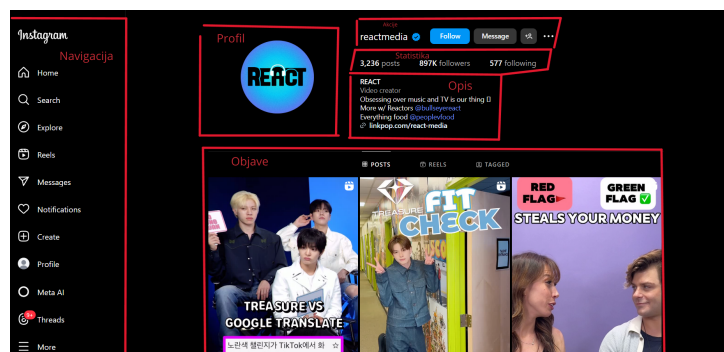
4.4.4. Ostale tehnologije

Za realizaciju praktičnog dijela korišteno je nekoliko tehnologija koje su omogućile razvoj web stranice i inteligentnog asistenta. Odabir tehnologija temelji se na njihovoj popularnosti, pouzdanosti i sposobnosti da zadovolje zahtjeve ovog rada. U nastavku su detaljnije opisani korišteni alati i tehnologije.

4.4.4.1. React

Svijet danas ne funkcionira bez mobilnih i web aplikacija. Sve se orijentira prema digitalizaciji, od rezerviranja hrane do naručivanja vožnje u taksiju. Vrlo je bitno napraviti vizualno primamljivu aplikaciju a React je upravo jedna od popularnijih frontend biblioteka koja to omogućava. React je temeljen na JavaScriptu i široko se koristi u web razvoju. U usporedbi s drugim tehnologijama, React je nova tehnologija, a osnovan je od strane Jordan Walkea 2011. godine (softverski inženjer u Facebooku). Reactova popularnost dolazi radi nekoliko razloga [37]:

- jednostavno stvaranje dinamičkih aplikacija,
- korištenje Virtual DOM čime se aplikacije brže izrađuju (ažurira komponente samo koje su promijenjene),
- podjela na komponente - jedna aplikacija se sastoji od više komponenti koje se mogu koristiti kroz cijelu aplikaciju (što smanjuje vrijeme razvoja aplikacije),
- mala krivulja učenja - React je jednostavan za učenje jer kombinira HTML i JavaScript koncepte,
- koristi se za razvoj web i mobilnih aplikacija - React Native - za mobilne aplikacije,
- jednostavno otklanjanje grešaka - Facebook je izdao Chromeovo proširenje koje se može koristiti za otklanjanje pogrešaka u React aplikacijama.



Slika 12: React komponenta

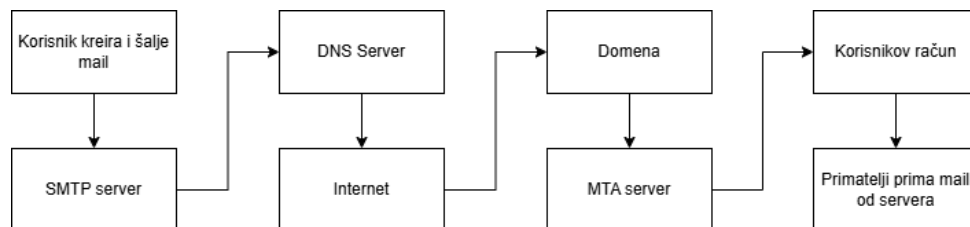
Jedna zanimljivost vezana uz React programere, promatrana na dvije potpuno različite lokacije, otkriva da su indijski developeri za isti posao plaćeni i do deset puta manje (podaci

za 2021. godinu). To jasno odražava razliku u životnom standardu i ekonomskim potrebama pojedinih zemalja [37].

- Prosječna plaća za početnog React developera u SAD-u iznosi oko 87000 USD godišnje,
- Prosječna plaća za početnog React developera u Indiji iznosi oko 8000 USD godišnje.

4.4.4.2. SMTP

SMTP (eng. Simple Mail Transfer Protocol) je TCP/IP protokol koji se koristi za slanje i primanje epošte preko mreže kao što je internet. Neki od primjera klijenata koji koriste SMTP su Gmail, Microsoft Outlook, Yahoo Mail i Apple Mail [38].



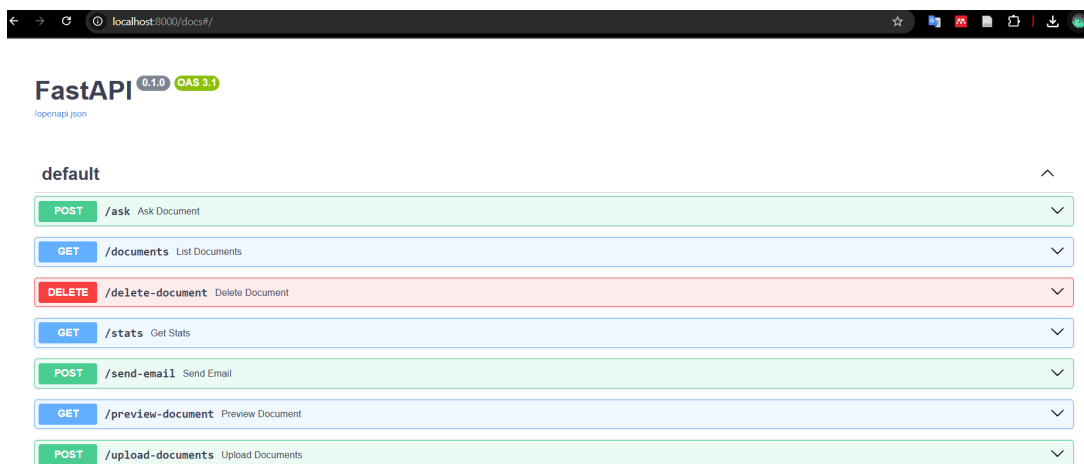
Slika 13: Proces dostavljanja emaila; Izvor

Prethodna slika prikazuje proces slanje epošte kroz SMTP protokol. Proces se odvija prema sljedećim koracima [38]:

1. Korisnik piše i šalje email npr. putem web aplikacije,
2. Email odlazi na SMTP poslužitelj koji je konfiguriran za primanje poruka i njihovo prosljeđivanje,
3. Nakon toga SMTP server kontaktira DNS server kako bi pronašao IP adresu poslužitelja primatelja. DNS prevodi adresu e-mail domene u IP adresu,
4. Kada se sazna gdje poruka ide, ona putuje kroz internet prema odredišnom serveru,
5. Email stiže na domenu primatelja i njen MTA (eng. Mail Transfer Agent) poslužitelj, koji prima poruku i pohranjuje u korisnikov sandučić,
6. Email je pohranjen na serveru primatelja i čeka da ga korisnik preuzme,
7. Primatelj može dohvatiti poruku putem IMAP (ostaje na serveru, pristup s više uređaja) ili POP (skida se lokalno, briše se sa servera) načina.

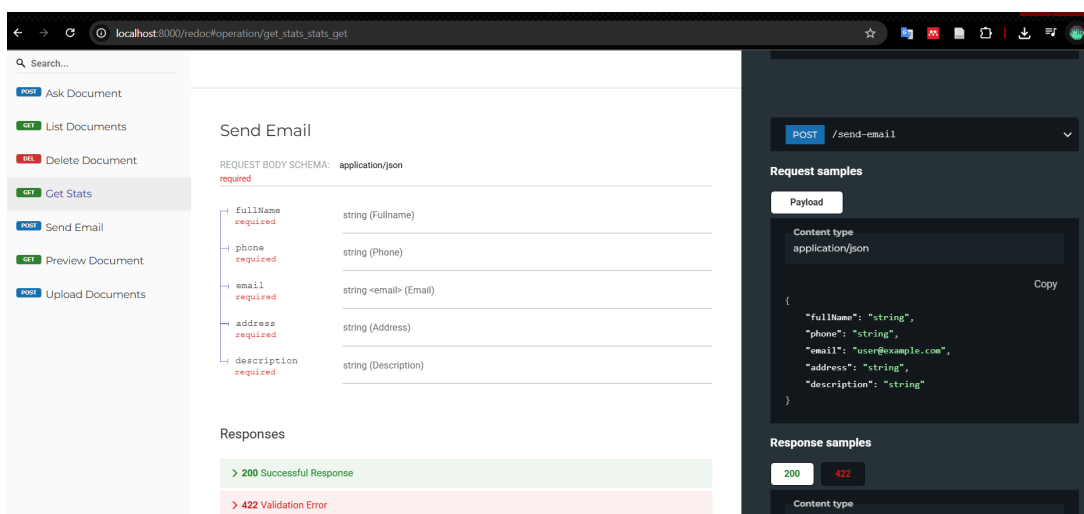
4.4.4.3. FastAPI

FastAPI je moderan, brz, visokoučinkovit web framework za izgradnju API-ja s Pythonom. Vrlo je brz (u rangu s NodeJS-om i Go-om) i omogućuje brzo kodiranje i pojavljivanje manjeg broja grešaka. Dosta je intuitivan i jednostavan za korištenje i učenje. FastAPI podržava CRUD (eng. Create, Read, Update, Delete) operacije koje je moguće koristiti kroz ključne riječi HTTP metoda (GET, POST, PUT, DELETE). Za pokretanje FastAPI koristi port 8000 i može se testirati putem preglednika ili alata poput Postmana. 8000/docs putanja omogućuje pristup Swagger UI sučelju koje omogućuje pregled i testiranje API-ja [39].



Slika 14: Interaktivna dokumentacija (OpenAPI) u Swagger stilu

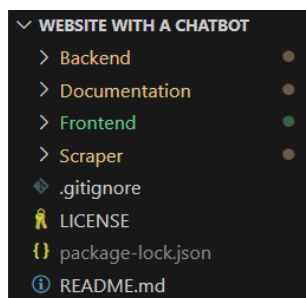
Ova dokumentacija omogućuje testiranje svih dostupnih API poziva izravno kroz preglednik. Alternativno, dostupna je ReDoc dokumentacija na ruti /redoc, koja prikazuje API specifikaciju na čitljiviji način, ali bez mogućnosti testiranja. Također, ruta /openapi.json prikazuje OpenAPI specifikaciju u JSON formatu, što je korisno za automatsko generiranje klijenata ili integraciju s alatima poput Postmana.



Slika 15: ReDoc prikaz dokumentacije

5. Praktični dio rada

Praktični dio rada se sastoji od izrade web stranice i izrade inteligentnog asistenta temeljenog na RAG tehnologiji. Frontend aplikacije je izrađen pomoću biblioteke React, pri čemu je korisničko sučelje strukturirano korištenjem JSX sintakse. Backend aplikacije razvijen je korištenjem Python web frameworka FastAPI, koji je poznat po svojoj jednostavnosti korištenja. Za izradu inteligentnog asistenta napravljen je scraper podataka s kojim je izrađen skup podataka koji je poslužio za rad inteligentnog asistenta.



Slika 16: Struktura praktičnog dijela rada

Struktura praktičnog dijela organizirana je na način da postoje 4 glavna dijela: backend, frontend, scraper i dokumentacija. Ideja ovakvog pristupa je da sve što je vezano za ovaj rad bude dostupno na jednom mjestu, čime se olakšava razumijevanje ovog rada. Prije početka rada potrebno je instalirati Python, NodeJS + npm, beautifulsoup4, fastapi, uvicorn, langchain, openai i requests.

Za instalaciju i konfiguraciju LaTeX-a u lokalnom okruženju unutar VS Code-a koristan je bio sljedeći video <https://www.youtube.com/watch?v=41yHIQ14VM8>. Za prikaz koda u dokumentu korištena je biblioteka minted, koja omogućuje sintaksno obojeni ispis izvornog koda. Budući da minted koristi vanjski program pygmentize, potrebno je omogućiti `--shell-escape` prilikom kompilacije. To se postiže izmjenom postavki u datoteci settings.json, gdje se dodaje odgovarajuća konfiguracija. CTRL + Shift + P (za otvaranje prozora).

```
1 {
2   "explorer.confirmDelete": false,
3   "php.validate.executablePath": "",
4   "latex-workshop.latex.tools": [{
5     "name": "pdflatex",
6     "command": "pdflatex",
7     "args": [
8       "--shell-escape",
9       "-interaction=nonstopmode",
10      "-synctex=1",
11      "%DOC%"
12    ]
13  }],
14   "latex-workshop.latex.recipes": [{
15     "name": "pdflatex",
16     "tools": ["pdflatex"]
17  }]
18 }
```

Isječak koda 1: settings.json

5.1. Struktura web stranice

Sljedeća tablica prikazuje pregled stranica web aplikacije te pripadajuće komponente koje su na njima implementirane. Svaka stranica ima svoju svrhu čime se osigurava intuitivna navigacija.

Tablica 5: Struktura stranica i njihovih komponenti

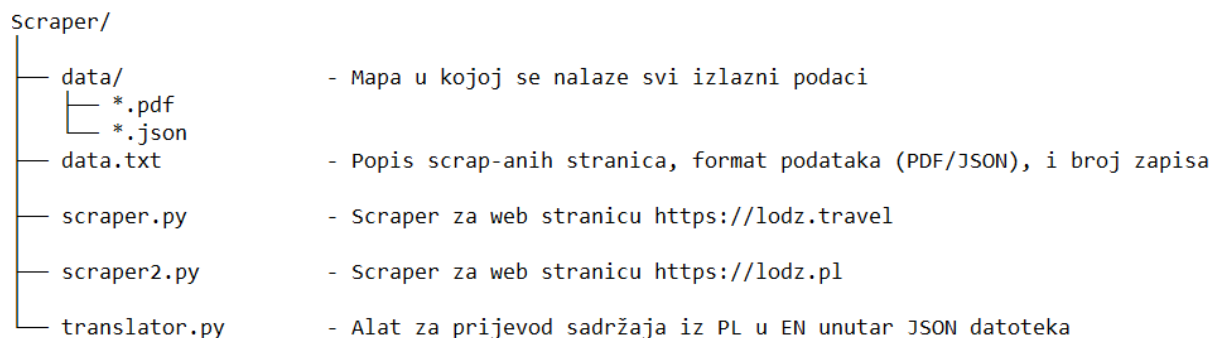
Stranica / Element	Komponente	Opis
Početna	Slider, Motivacija, Kratko o Łódžu, Drugi gradovi	Uvodna stranica s informacijama i poveznicama prema ostalim dijelovima stranice.
O Łódžu	Dodatno o gradu, Iskustva bivših studenata	Informacije o gradu i iskustvima Erasmus+ studenata.
Drugi gradovi	Pregled gradova	Pregled ostalih gradova Poljske koje je moguće posjetiti.
Pitanja	Forma za postavljanje pitanja	Korisnik postavlja pitanje ako AI asistent ne ponudi tražene informacije.
Dokumenti	Pregled, dodavanje i brisanje dokumenata za RAG	Sekcija za prikaz i pregled dokumenata koji služe kako bi se odgovorilo na pitanja korisnika.
AI asistent (pop-up)	Sučelje za razgovor	Interaktivni chat s korisnikom; otvara se klikom na ikonu asistenta.

[Napomena] - Svaka stranica sadrži obavezne elemente kao što su podnožje (eng. footer) i zaglavlje (eng. header).

5.2. Prikupljanje podataka

Prikupljanje podataka započelo je kroz razgovor s profesoricom umjetne inteligencije koju sam upoznao tijekom Erasmus+ razmjene. Profesorica je bila izuzetno korisna osoba za ovaj rad jer je rođena i živi u gradu Łódź. Ukratko sam joj predstavio temu rada, a ona mi je preporučila nekoliko relevantnih internetskih izvora.

Za kreiranje skupa podataka korišten je Python zajedno s bibliotekom BeautifulSoup. Scraper folder je potpuno samostalna cijelina i neovisna je o backend i frontend dijelu. Unutar scraper foldera postoji data folder u kojemu su spremljeni izlazni podaci koji su u dva oblika (JSON i pdf). Data.txt predstavlja kratki sažetak koji opisuje o kakvim se podacima radi, koliko ih ima i odakle su preuzeti. Pošto su korištene dvije stranice kao izvori podataka, potrebno je bilo napraviti dva scrapera od kojih je samo za jedan u nastavku detaljnije opisana implementacija. Osim toga pošto su stranice izvorno na poljskom jeziku bilo je potrebno napraviti skriptu koja prevodi JSONE na engleski jezik radi lakšeg razumijevanja JSON-a kako ljudima pa tako i AI modelu.



Slika 17: Struktura scraper foldera

Sljedeća tablica prikazuje pregled korištenih izvora, broj zapisa, naziv rezultirajuće datoteke i alate korištene za obradu podataka:

Web izvor	Veličina skupa podataka	Naziv datoteke	Korišteni alati
https://lodz.pl/	1000 zapisa	lodz_articles	scraper2.py, translator.py
https://lodz.travel/turystyka/co-zobaczyc/	185 zapisa	lodz_destinations	scraper.py, translator.py
https://polskapogodzinach.pl/lodz-atrakcje/	52 stranice	lodz_attractions	ručno konvertirano u PDF

Tablica 6: Korišteni izvori podataka i alati za obradu

Sljedeća metoda dohvaća HTML sadržaj stranice. Prima URL i vraća BeautifulSoup objekt za lakšu obradu HTML-a. Postavlja User-Agent kako bi se izbjeglo blokiranje od strane servera. U slučaju do pogreške (npr. timeout) ispisuje poruku i vraća None.

```

1  import requests
2  from bs4 import BeautifulSoup
3  from urllib.parse import urljoin
4  import json
5  import time

6  BASE_URL = "https://lodz.travel"
7  visited_urls = set()

8  def get_soup(url):
9      print(f"Fetching: {url}")
10     try:
11         res = requests.get(url, headers={
12             "User-Agent": "Mozilla/5.0"
13         })
14         res.encoding = 'utf-8'
15         return BeautifulSoup(res.text, 'html.parser')

```

```

16 except Exception as e:
17     print(f"Error fetching page: {e}")
18     return None

```

Isječak koda 2: Scraper 1/6

Ova metoda rekurzivno prolazi kroz listu i sve li stavke u njoj. Za svaku pronađenu stranicu iz izbornika izdvaja se title i href te se stvara struktura koja sadrži title, url, content i subcategories.

```

1 def parse_menu(ul_tag, level=1):
2     data = []
3     for li in ul_tag.find_all("li", recursive=False):
4         a_tag = li.find("a", recursive=False)
5         if not a_tag:
6             continue
7         title = a_tag.text.strip()
8         href = urljoin(BASE_URL, a_tag["href"])
9         print(f"' ' * (level-1))({level}) {title} -> {href}")
10        node = {
11            "title": title,
12            "url": href,
13            "content": "",
14        }
15        sub_ul = li.find("ul")
16        if sub_ul:
17            node["subcategories"] = parse_menu(sub_ul, level + 1)
18        data.append(node)
19    return data

```

Isječak koda 3: Scraper 2/6

Sljedeća metoda dohvaća sadržaj za svaku stranicu. Ulazi u svaku stranicu u hijerarhiji i izdvaja tekst iz diva s klasom ce-bodytext. Sprema pročišćeni tekst u item["content"] te traži dodatne poveznice unutar sadržaja i dodaje ih kao subcategories i rekurzivno ih obađuje.

```

1 def fill_content_recursively(data):
2     for item in data:
3         if item["url"] in visited_urls:
4             continue
5         visited_urls.add(item["url"])
6
7         print(f"\nRetrieving content for: {item['title']}")
8         soup = get_soup(item["url"])
9         if not soup:
10            continue
11
12        content_div = soup.select_one("div.ce-bodytext")
13        if content_div:
14            item["content"] = content_div.get_text(separator="\n", strip=True)
15            print(f"Content retrieved successfully ({len(item['content'])}
16                  ↪ characters)")
17
18        sub_links = []
19        for a in content_div.find_all("a", href=True):
20            href = urljoin(BASE_URL, a["href"])
21            if BASE_URL in href and href not in visited_urls:
22                title = a.get_text(strip=True)
23                if title:

```

```

20         sub_links.append({
21             "title": title,
22             "url": href,
23             "content": ""
24         })

25     if sub_links:
26         print(f"Found {len(sub_links)} internal subpages inside:
27             ↳ {item['title']}")
28         item["subcategories"] = sub_links
29         fill_content_recursively(sub_links)

30     if "subcategories" in item:
31         fill_content_recursively(item["subcategories"])

```

Isječak koda 4: Scraper 3/6

Ova metoda broji sve stavke (stranice i podstranice) u hijerarhiji podataka. Koristi se za statistiku na kraju izvršavanja.

```

1 def count_total_items(data):
2     count = 0
3     for item in data:
4         count += 1
5         if "subcategories" in item:
6             count += count_total_items(item["subcategories"])
7     return count

```

Isječak koda 5: Scraper 4/6

Ova metoda pokreće cijeli proces. Kreće od glavne stranice navigacije gdje se dohvaća struktura linkova pomoću parse menu funkcije. Zatim se rekurzivno dohvaća sadržaj sa svih stranica pomoću fill content recursively funkcije. Na kraju svi podaci se spremaju u JSON datoteku i ispisuje se koliko je ukupno stranica pronađeno i spremljeno.

```

1 def main():
2     start_url = "https://lodz.travel/turystyka/"
3     soup = get_soup(start_url)
4     aside_nav = soup.select_one("ul.aside-nav")
5     if not aside_nav:
6         print("Navigation not found! Check the URL or HTML structure.")
7         return
8     print("Parsing main navigation...")
9     structured_data = parse_menu(aside_nav)
10    print("\nFetching content from all pages...")
11    fill_content_recursively(structured_data)
12    with open("lodz_content.json", "w", encoding="utf-8") as f:
13        json.dump(structured_data, f, indent=2, ensure_ascii=False)
14    total = count_total_items(structured_data)
15    print(f"\nTotal records extracted: {total}")
16    print("Done! Data saved to: lodz_content.json")
17 if __name__ == "__main__":
18     main()

```

Isječak koda 6: Scraper 5/6

Ova python skripta učitava JSON s člancima na poljskom jeziku i prevodi ih na engleski koristeći biblioteku deep-translator. Funkcija rekurzivno pronalazi kroz sve stavke i podkatego-

rije u JSON strukturi. Napredak prevođenja ispisuje se u konzoli, a prevedeni podaci spremaju se u novu .json datoteku.

```
1 import json
2 from deep_translator import GoogleTranslator

3 INPUT_FILE = "lodz_articles_pl.json"
4 OUTPUT_FILE = "lodz_articles_en.json"

5 def translate_text(text, src="pl", target="en"):
6     try:
7         return GoogleTranslator(source=src, target=target).translate(text)
8     except Exception as e:
9         return f"[Translation Error] {text}"

10 def translate_node(node, counter, total):
11     if "title" in node:
12         node["title"] = translate_text(node["title"])
13     if "content" in node and node["content"]:
14         node["content"] = translate_text(node["content"])

15     counter[0] += 1
16     print(f"[{counter[0]}/{total}] translated")

17     if "subcategories" in node:
18         for sub in node["subcategories"]:
19             translate_node(sub, counter, total)

20 def count_nodes(data):
21     count = 0
22     for item in data:
23         count += 1
24         if "subcategories" in item:
25             count += count_nodes(item["subcategories"])
26     return count

27 def main():
28     print("Loading JSON...")
29     with open(INPUT_FILE, "r", encoding="utf-8") as f:
30         data = json.load(f)

31     total = count_nodes(data)
32     print(f"Translating {total} entries...")

33     counter = [0]
34     for entry in data:
35         translate_node(entry, counter, total)

36     print("Saving translated JSON...")
37     with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
38         json.dump(data, f, ensure_ascii=False, indent=2)

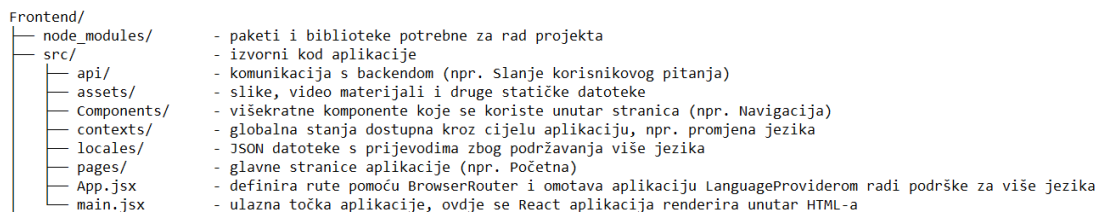
39     print(f"Done! Translated file saved as: {OUTPUT_FILE}")

40 if __name__ == "__main__":
41     main()
```

Isječak koda 7: Scraper 6/6

5.3. Korisničko sučelje (Frontend)

Na sljedećoj slici može se vidjeti struktura frontend dijela. U okviru frontend dijela prikazani su samo odabrani dijelovi koda i komponente zbog njihove opsežnosti. U slučaju interesa za konkretnim dijelom, izvorni kod je dostupan na Github repozitoriju koji je naveden u zaključku rada.



Slika 18: Struktura frontend dijela

Funkcija `askAssistant` šalje korisničko pitanje AI asistentu putem POST zahtjeva na backend. Automatski generira i koristi jedinstveni ID sesije, koji se sprema u `localStorage`. Na kraju vraća JSON odgovor s asistentskim odgovorom ili javlja grešku ako zahtjev ne uspije.

```
1 import { v4 as uuidv4 } from "uuid";

2 let sessionId = localStorage.getItem("chat_session_id");
3 if (!sessionId) {
4     sessionId = uuidv4();
5     localStorage.setItem("chat_session_id", sessionId);
6 }

7 export async function askAssistant(question, files = []) {
8     const formData = new FormData();
9     formData.append("question", question);
10    formData.append("session_id", sessionId);

11    if (files && files.length > 0) {
12        files.forEach(file => formData.append("pdfs", file));
13    }

14    const response = await fetch("http://localhost:8000/ask", {
15        method: "POST",
16        body: formData,
17    });

18    if (!response.ok) {
19        throw new Error(`Request failed: ${response.statusText}`);
20    }

21    const data = await response.json();
22    return data;
23 }
```

Isječak koda 8: Chat.jsx

Funkcija `sendContactForm` šalje podatke iz kontakt forme na backend rutu `/send email` koristeći POST zahtjev s JSON tijelom. Koristi se za slanje emailova s korisničkim podacima (ime, email, poruka itd.) na server putem FastAPI endpointa.

```
1 export async function sendContactForm(formData) {
2   const response = await fetch("http://localhost:8000/send-email", {
3     method: "POST",
4     headers: {
5       "Content-Type": "application/json",
6     },
7     body: JSON.stringify(formData),
8   });
9   if (!response.ok) {
10    const data = await response.json();
11    throw new Error(data.error || "Greška prilikom slanja e-maila.");
12  }
13  return response.json();
14 }
```

Isječak koda 9: Contact.jsx

Sljedeća funkcija dohvaća popis trenutno pohranjenih dokumenata sa servera. Koristi GET zahtjev prema `/documents`.

```
1 export async function fetchDocuments() {
2   const res = await fetch("http://localhost:8000/documents");
3   if (!res.ok) throw new Error("Failed to fetch documents");
4   return await res.json();
}
```

Isječak koda 10: Document.jsx 1/5

Ova funkcija dohvaća statistiku o dokumentima s rute `/stats`. Vraća podatke poput broja dokumenta, broja ulomka i vremena zadnjeg dodavanja. Također koristi GET i vraća grešku ako ne uspije.

```
1 export async function fetchStats() {
2   const res = await fetch("http://localhost:8000/stats");
3   if (!res.ok) throw new Error("Failed to fetch stats");
4   return await res.json();
}
```

Isječak koda 11: Document.jsx 2/5

Sljedeća funkcija briše dokument prema nazivu koristeći DELETE zahtjev. Naziv datoteke se enkodira kako bi bio siguran za URL.

```
1 export async function deleteDocument(filename) {
2   const res = await fetch(`http://localhost:8000/delete-document?filename=${encodeURIComponent(filename)}`, {
3     method: "DELETE",
4   });
5   if (!res.ok) throw new Error("Failed to delete document");
6   return await res.json();
7 }
```

Isječak koda 12: Document.jsx 3/5

Sljedeća funkcija se koristi za slanje jednog ili više dokumenata (PDF/JSON). Kreira FormData i uključuje ID sesije. Ako server ne prihvati datoteke, funkcija vraća grešku.

```
1 export async function uploadDocuments(files, sessionId) {
2   const formData = new FormData();
3   formData.append("session_id", sessionId);
4   files.forEach(file => formData.append("pdfs", file));

5   const res = await fetch("http://localhost:8000/upload-documents", {
6     method: "POST",
7     body: formData,
8   });

9   if (!res.ok) throw new Error("Failed to upload documents");
10  return await res.json();
11 }
```

Isječak koda 13: Document.jsx 4/5

Posljednja funkcija dohvaća sadržaj dokumenta (npr. JSON) za pregled. Koristi GET zahtjev prema /preview document. Vraća sadržaj u čitljivom formatu ili javlja grešku.

```
1 export async function previewDocument(filename) {
2   const res = await fetch(`http://localhost:8000/preview-document?filename=${encodeURIComponent(filename)}`);
3   if (!res.ok) throw new Error("Failed to preview document");
4   return await res.json();
5 }
```

Isječak koda 14: Document.jsx 5/5

About komponenta prikazuje sekciju s videozapisom i informacijama od gradu Lodzu. Koristi useContext i LanguageContext čime se dohvaćaju prijevodi teksta na temelju trenutno odabranog jezika (translations.aboutPage). Nadalje ugrađeni iframe iz Google Drive-a prikazuje video o Lodzu. Postavljen je na širinu 100 posto i visinu 400px. Za taj video bilo je potrebno odvojiti jedan dan kako bi se napisali i složili engleski titlovi kako bi i on mogao biti popraćen od strane nekoga ko ne zna hrvatski jezik. Nastavno se prikazuje naslov i dva odlomka teksta preuzeta iz konteksta jezika (paragraph1 i paragraph2). Na kraju su prikazane tri ikone s pripadajućim opisima.

```
1 import React, { useContext } from 'react';
2 import { LanguageContext } from '../contexts/LanguageContext';
3 import './About.css';

4 const About = () => {
5   const { translations } = useContext(LanguageContext);

6   return (
7     <>
8       <section className='about'>
9         <div className='video'>
10           <iframe
11             src="https://drive.google.com/file/d/18k77Ntbqf1-yN5F-VsQ9CnGWgzZe6gm/_
12             ↪ preview"
13             width="100%"
14             height="400"
```

```

14     allow="autoplay"
15     allowFullScreen
16     title="About Łódź Video"
17     style={{ border: "none", borderRadius: "12px" }}
18   ></iframe>
19 </div>

20 <div className='content'>
21   <h3 id='vise-o-lodzu'>{translations.aboutPage.subTitle}</h3>

22   <p>{translations.aboutPage.paragraph1}</p>
23   <p>{translations.aboutPage.paragraph2}</p>

24   <div className="icons-container">
25     <div className='icons'>
26       <i className='fas fa-shield-alt'></i>
27       <span>{translations.aboutPage.safe}</span>
28     </div>
29     <div className='icons'>
30       <i className='fas fa-leaf'></i>
31       <span>{translations.aboutPage.clean}</span>
32     </div>
33     <div className='icons'>
34       <i className='fas fa-building'></i>
35       <span>{translations.aboutPage.modern}</span>
36     </div>
37   </div>
38 </div>
39 </section>
40 </>
41 )
42 }
43 export default About;

```

Isječak koda 15: About.jsx komponenta

Sljedeća datoteka postavlja kontekst za jezik aplikacije koristeći React Context API. Omogućuje da cijela aplikacija zna koji jezik je trenutno aktivan i da prema tome prikazuje prijevode. Započetak se učitavaju dva JSON-a (hr.json i en.json). State za jezik postavlja engleski kao početni jezik (useState('en')), dok se setLanguage koristi za promjenu jezika. Nadalje ako je odabrani jezik hr, koristi se hr.json, u suprotnom en.json. Funkcija toggleLanguage omogućuje promjenu jezika pozivom funkcije toggleLanguage('hr'). LanguageContext.Provider omogućuje da djeca komponente (children) pristupe vrijednostima language, toggleLanguage i translations.

```

1 import React, { createContext, useState } from 'react';
2 import hr from '../locales/hr.json';
3 import en from '../locales/en.json';

4 export const LanguageContext = createContext();

5 export const LanguageProvider = ({ children }) => {
6   const [language, setLanguage] = useState('en');

7   const translations = language === 'hr' ? hr : en;

8   const toggleLanguage = (lang) => {
9     setLanguage(lang);
10  };

```

```

11   return (
12     <LanguageContext.Provider value={{ language, toggleLanguage, translations }}>
13       {children}
14     </LanguageContext.Provider>
15   );
16 };

```

Isječak koda 16: LanguageContext.jsx

Sljedeća komponenta predstavlja glavnu početnu stranicu aplikacije i prikazuje sve ključne sekcije. Koristi višekratne komponente poput Navbar, Slider, Motivation, Cities, Help i Footer za izgradnju cjelovitog prikaza. Stranica uključuje i pomoćne elemente poput gumba za povratak na vrh (ScrollToTopButton).

```

1  import React from 'react'
2  import Navbar from '../Components/Navbar'
3  import Footer from '../Components/Footer'
4  import Motivation from '../Components/Motivation'
5  import Slider from '../Components/Slider'
6  import AboutLodz from '../Components/AboutLodz'
7  import Cities from '../Components/Cities'
8  import Help from '../Components/Help'
9  import Icon from '../Components/AIAssistant/Icon'
10 import ScrollToTopButton from '../Components/ScrollToTopButton'
11
12 export default function() {
13   return(
14     <>
15       <Navbar/>
16       <Slider/>
17       <Motivation/>
18       <AboutLodz/>
19       <Cities/>
20       <Help/>
21       <ScrollToTopButton/>
22       <Footer/>
23     </>
24   )
25 }

```

Isječak koda 17: Stranica Home

Ova glavna React komponenta postavlja strukturu aplikacije koristeći BrowserRouter za upravljanje rutama. Omotana je u LanguageProvider kako bi sve podstranice imale pristup jeziku i prijevodima. Unutar Routes definirane su rute prema komponentama Home, About, Cities, Question i Documents, dok je komponenta Icon prikazana globalno - neovisno o stranici (kako bi popup prozor asistenta bio i dalje otvoren iako se prelazi na druge stranice).

```

1  import React from 'react'
2  import {BrowserRouter, Routes, Route,} from 'react-router-dom';
3  import { LanguageProvider } from '../contexts/LanguageContext';
4  import Home from './pages/Home'
5  import About from './pages/About'
6  import Cities from './pages/Cities'
7  import Question from './pages/Question'
8  import Documents from './pages/Documents';
9  import Icon from '../Components/AIAssistant/Icon';

```

```

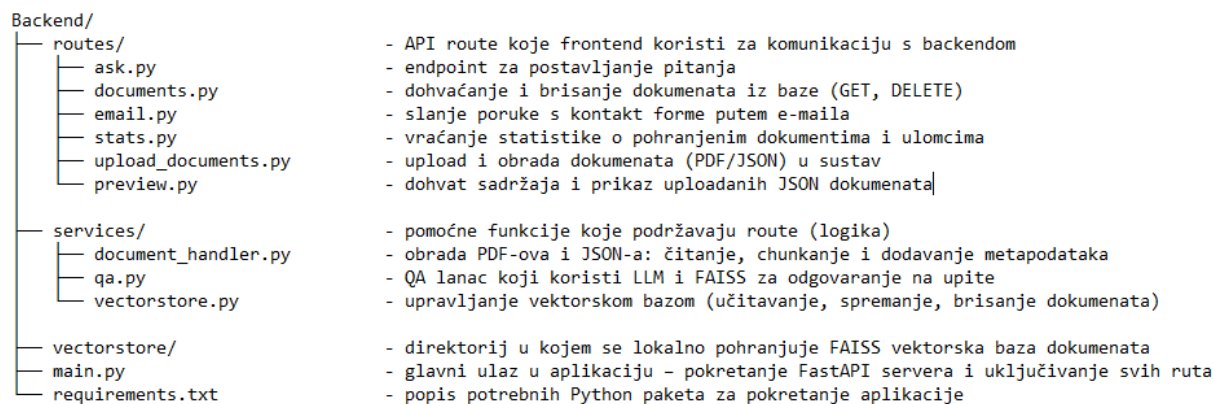
10 const App = () => {
11   return (
12     <LanguageProvider>
13     <BrowserRouter>
14     <Icon />
15     <Routes>
16       <Route index element={<Home />} />
17       <Route path='/home' element={<Home />} />
18       <Route path='/about' element={<About />} />
19       <Route path='/cities' element={<Cities />} />
20       <Route path='/question' element={<Question />} />
21       <Route path='/documents' element={<Documents />} />
22     </Routes>
23   </BrowserRouter>
24 </LanguageProvider>
25 );
26 };
27 export default App

```

Isječak koda 18: App.jsx

5.4. Poslužiteljski dio (Backend)

Na sljedećoj slici može se vidjeti struktura backend dijela. Prikazane su sve stvari koje su od značaja za backend dio aplikacije.



Slika 19: Struktura backend dijela

Env datoteka sadrži konfiguracijske varijable koje se koriste u aplikaciji. Promjenjive vrijednosti uključuju API ključeve za OpenAI i Groq modele (OPENAI API KEY, GROQ API KEY) kao i HF API KEY za Hugging Face modele. Za slanje poruka putem kontakt forme koriste se Gmail podaci: Gmail user je adresa s koje se poruke šalju, Gmail password je aplikacijska lozinka za autentifikaciju i Gmail receiver je adresa primatelja. Model koji koristi Hugging Face definiran je kroz varijablu HUGGINGFACEHUB EMBEDDINGS MODEL NAME.

```

1 OPENAI_API_KEY=
2 GROQ_API_KEY=
3
4 GMAIL_USER=
5 GMAIL_PASSWORD=
6 GMAIL_RECEIVER=
7
8 HF_API_KEY=
9 HUGGINGFACEHUB_EMBEDDINGS_MODEL_NAME=

```

Slika 20: Sadržaj env datoteke

5.4.1. Server

Main predstavlja glavnu datoteku koja pokreće FastAPI aplikaciju. Učitava se konfiguracija iz .env datoteke, omogućuje se CORS pristup kako bi frontend i backend mogli komunicirati bez ograničenja, te se u aplikaciju uključuju sve definirane rute za pitanja, dokumente, statistiku i email.

```

1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from dotenv import load_dotenv
4
5 from routes import ask, documents, stats, email
6
7 load_dotenv()
8
9 app = FastAPI()
10
11 app.add_middleware(
12     CORSMiddleware,
13     allow_origins=["*"],
14     allow_credentials=True,
15     allow_methods=["*"],
16     allow_headers=["*"],
17 )
18
19 app.include_router(ask.router)
20 app.include_router(documents.router)
21 app.include_router(stats.router)
22 app.include_router(email.router)

```

Isječak koda 19: Main.py

5.4.2. Servisi

Sljedeći dio koda upravlja vektorskom bazom podataka koristeći FAISS i HuggingFaceEndpointEmbeddings. Omogućuje učitavanje vektorske baze podataka, dodavanje novih dokumenata te brisanje dokumenata na temelju imena datoteke. Vektorska baza podataka se lokalno pohranjuje u direktorij vectorstore. Započetak se dohvaća API ključ iz .env datoteke (HF_API_KEY), ako ključ ne postoji vraća se greška. Nakon toga se vraća instanca HuggingFaceEndpointEmbeddings za model sentence-transformers/all-MiniLM-L6-v2 (koristi se u svim ostalim funkcijama gdje je potrebno raditi s vektorima).

```

1 import os
2 from langchain_community.vectorstores import FAISS
3 from langchain_huggingface import HuggingFaceEndpointEmbeddings
4 from dotenv import load_dotenv

5 load_dotenv()

6 VECTORSTORE_DIR = "vectorstore"

7 def get_embeddings():
8     api_key = os.getenv("HF_API_KEY")

9     if not api_key:
10         raise ValueError("HF_API_KEY is not set in the environment.")

11     return HuggingFaceEndpointEmbeddings(
12         model="sentence-transformers/all-MiniLM-L6-v2",
13         huggingfacehub_api_token=api_key
14     )

```

Isječak koda 20: Vectorstore.py 1/4

Nadalje provjerava se postoji li direktorij vectorstore, ako ne postoji, vraća se `FileNotFoundError`. Učitava se FAISS baza pomoću prethodno definiranih embeddinga (koristi se kada se želi čitati postojeće dokumente)

```

1 def load_vectorstore():
2     if not os.path.exists(VECTORSTORE_DIR):
3         raise FileNotFoundError("Vectorstore folder not found.")

4     embeddings = get_embeddings()
5     db = FAISS.load_local(VECTORSTORE_DIR, embeddings,
6         ↪ allow_dangerous_deserialization=True)
7     return db

```

Isječak koda 21: Vectorstore.py 2/4

Sljedeći dio koda dohvaća embeddinge i ako direktorij vectorstore postoji: učitava bazu, dodaje dokumente ako ih ima, i sprema ažuriranu bazu. Ako baza ne postoji a ako ima dokumenata, kreira se nova baza i sprema se (koristi se kod prvog unosa dokumenta).

```

1 def get_or_create_vectorstore(all_docs):
2     embeddings = get_embeddings()

3     if os.path.exists(VECTORSTORE_DIR):
4         db = FAISS.load_local(VECTORSTORE_DIR, embeddings,
5             ↪ allow_dangerous_deserialization=True)
6         if all_docs:
7             print(f"Adding {len(all_docs)} documents...")
8             db.add_documents(all_docs)
9             db.save_local(VECTORSTORE_DIR)
10            print(f"Updated vectorstore. Total docs: {len(db.docstore._dict)}")
11        elif all_docs:
12            print(f"Creating vectorstore with {len(all_docs)} documents...")
13            db = FAISS.from_documents(all_docs, embeddings)
14            db.save_local(VECTORSTORE_DIR)
15            print("Vectorstore created and saved.")
16        else:

```

```

16         raise ValueError("No documents to index and no vectorstore exists.")
17     return db

```

Isječak koda 22: Vectorstore.py 3/4

Sljedeći dio koda učitava vektorsku bazu podataka nakon čega filtrira sve dokumente čiji source nije jednak filename. Ako se broj dokumenata nije promijenio to znači da taj dokument nije postojao. Inače se kreira nova baza samo s preostalim dokumentima, sprema se i vraća True (koristi se kada korisnik želi ukloniti određeni dokument iz baze).

```

1 def delete_file_from_vectorstore(filename):
2     try:
3         db = load_vectorstore()
4         except FileNotFoundError:
5             print("No vectorstore to delete from.")
6             return False
7
8         original_count = len(db.docstore._dict)
9         remaining_docs = [
10             doc for doc in db.docstore._dict.values()
11             if doc.metadata.get("source") != filename
12         ]
13
14         if len(remaining_docs) == original_count:
15             print(f"No document found with source: {filename}")
16             return False
17
18         print(f"Deleting document: {filename}")
19         embeddings = get_embeddings()
20         new_db = FAISS.from_documents(remaining_docs, embeddings)
21         new_db.save_local(VECTORSTORE_DIR)
22         print("Vectorstore updated after deletion.")
23         return True

```

Isječak koda 23: Vectorstore.py 4/4

Memory store predstavlja globalni riječnik u kojem se za svaki korisnički session id pamti prethodna komunikacija (memorija razgovora). Samim time se omogućuje dati kontekst odgovorima jer se zna što je prethodno rečeno. U funkciji get conversation chain incijalizira se LLM iz GROQ API-ja, model koristi llama-3.1-8b-instant za brze odgovore i ključ se učitava iz .env datoteke. Nadalje se incijalizira memorija po korisniku i ako memorija za dani session id još ne postoji, kreira se nova. Samim time je omogućeno vođenje dijaloga s kontinuitetom odnosno ne počinje iznova pri svakom pitanju. Nadalje ako postoje dokumenti u FAISS bazi, pretvaraju se u retriever koji dohvaća najrelevantnije dijelove. Spaja se jezični model, pretraga i memorija u jedan lanac (ConversationalRetrievalChain). Za kraj ako vektorska baza podataka nije dostupna, koristi se obični lanac samo s memorijom i modelom (ConversationChain).

```

1 from langchain.memory import ConversationBufferMemory
2 from langchain.chains import ConversationalRetrievalChain, ConversationChain
3 from langchain_groq import ChatGroq
4 from langchain_community.vectorstores import FAISS
5 import os
6
7 memory_store = {}

```

```

7 def get_conversation_chain(db: FAISS, session_id: str):
8     llm = ChatGroq(
9         model="llama-3.1-8b-instant",
10        api_key=os.getenv("GROQ_API_KEY")
11    )
12
13    if session_id not in memory_store:
14        memory_store[session_id] = ConversationBufferMemory(
15            memory_key="chat_history",
16            return_messages=True
17        )
18    memory = memory_store[session_id]
19
20    if db and db.docstore._dict:
21        retriever = db.as_retriever(search_kwargs={"k": 3})
22        return ConversationalRetrievalChain.from_llm(
23            llm=llm,
24            retriever=retriever,
25            memory=memory,
26            verbose=True
27        )
28    else:
29        return ConversationChain(
30            llm=llm,
31            memory=memory,
32            verbose=True
33        )

```

Isječak koda 24: qa.py

Ovdje se prvo provjeravaju jesu li datoteke poslone nakon čega se kreiraju privremene mape za pohranu dokumenata gdje će se dokumenti privremeno spremiti lokalno prije obrade. Nakon toga iterira se kroz sve datoteke i gdje se svaki dokument privremeno zapisuje na disk (sadržaj se čita asinkrono). Zatim se provjerava tip dokumenta i ako je pdf, koristi se PyPDF-Loader iz Langchain-a za učitavanje sadržaja kao Document. Ako je .json, parsira se kao JSON objekt i pretvara u tekst koji se spremi kao jedan Document. Nastavno se svakom dokumentu dodaje source i točan UTC datum i vrijeme slanja (metapodaci). Za kraj sadržaj se dijeli u dijelove od 1000 znakova, s preklapanjem od 200 znakova. Time se čuva kontekst između dijelova.

```

1 import os
2 import tempfile
3 import json
4 from datetime import datetime
5 from fastapi import UploadFile
6 from langchain_community.document_loaders import PyPDFLoader
7 from langchain.docstore.document import Document
8 from langchain.text_splitter import RecursiveCharacterTextSplitter
9
10 async def process_documents(files: list[UploadFile]):
11     if not files:
12         return []
13
14     temp_dir = tempfile.TemporaryDirectory()
15     all_docs = []
16
17     for file in files:
18         temp_path = os.path.join(temp_dir.name, file.filename)

```

```

16     with open(temp_path, "wb") as f:
17         f.write(await file.read())

18     if file.filename.lower().endswith(".pdf"):
19         loader = PyPDFLoader(temp_path)
20         docs = loader.load()

21     elif file.filename.lower().endswith(".json"):
22         try:
23             with open(temp_path, "r", encoding="utf-8") as f:
24                 json_data = json.load(f)

25                 text = json.dumps(json_data, indent=2, ensure_ascii=False)
26                 docs = [Document(page_content=text, metadata={"source":
27                     ↪ file.filename})]
28             except Exception as e:
29                 print(f"Greška prilikom parsiranja JSON-a: {file.filename} -- {e}")
30                 continue

31     else:
32         print(f"Preskačem nepodržani format: {file.filename}")
33         continue

34     for doc in docs:
35         doc.metadata["source"] = file.filename
36         doc.metadata["uploaded_at"] = datetime.utcnow().isoformat()

37     splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
38         ↪ chunk_overlap=200)
39     split_docs = splitter.split_documents(docs)
40     all_docs.extend(split_docs)

41     return all_docs

```

Isječak koda 25: Document_handler.py

5.4.3. Ruta

Ova API ruta omogućuje korisniku da pošalje pitanje i jedinstveni ID sesije, a zatim dohvaća odgovor koristeći vektorsku bazu podataka i LLM model. Prvo se pokušava učitati lokalna vektorska baza, ako ne postoji, vraća se obavijest da dokumenti još nisu učitani. Ako je sve u redu, poziva se ConversationalRetrievalChain koji koristi kontekst prethodnog razgovora i vraća najrelevantniji odgovor. U slučaju greške, vraća se poruka s odgovarajućim statusom.

```

1  from fastapi import APIRouter, Form
2  from fastapi.responses import JSONResponse
3  from services.vectorstore import load_vectorstore
4  from services.qa import get_conversation_chain
5  import traceback

6  router = APIRouter()

7  @router.post("/ask")
8  async def ask_document(
9      question: str = Form(...),
10     session_id: str = Form(...)
11 ):
12     try:

```

```

13     db = load_vectorstore()
14     except FileNotFoundError:
15         return {"answer": "No documents are loaded yet. Please upload documents"}
16         ↪ first."}
17     except Exception as e:
18         print("Error loading vectorstore:", str(e))
19         traceback.print_exc()
20         return JSONResponse(status_code=500, content={"answer": "Error loading"}
21         ↪ vectorstore."})
22
23     try:
24         qa_chain = get_conversation_chain(db, session_id)
25         response = qa_chain.invoke({"question": question})
26
27         print("== CHAT MEMORY ==")
28         for msg in qa_chain.memory.chat_memory.messages:
29             role = msg.type
30             print(f"{role.upper()}: {msg.content}")
31
32         answer = response["answer"] if isinstance(response, dict) else response
33
34         return {
35             "answer": answer
36         }
37
38     except Exception as e:
39         print("Error during QA invocation:", str(e))
40         traceback.print_exc()
41         return JSONResponse(status_code=500, content={
42             "answer": "Sorry, there was an error processing your question. Please"}
43             ↪ try again later."
44         })

```

Isječak koda 26: ask.py

Nadalje su definirane dvije rute: jedna za dohvat popisa svih PDF dokumenata koji su trenutno indeksirani u vektorskoj bazi podataka, a druga za brisanje dokumenata po nazivu dokumenta. Ruta za brisanje vraća grešku ako dokument s tim imenom ne postoji u vektorskoj bazi podataka.

```

1  from fastapi import APIRouter, HTTPException
2  from services.vectorstore import load_vectorstore, delete_file_from_vectorstore
3
4  router = APIRouter()
5
6  @router.get("/documents")
7  def list_documents():
8      db = load_vectorstore()
9      sources = set()
10     for doc in db.docstore._dict.values():
11         if "source" in doc.metadata:
12             sources.add(doc.metadata["source"])
13     return {"documents": list(sources)}
14
15  @router.delete("/delete-document")
16  def delete_document(filename: str):
17     success = delete_file_from_vectorstore(filename)
18     if not success:
19         raise HTTPException(status_code=404, detail=f"No document found with source: {filename}")
20     return {"status": "success", "message": f"Deleted document: {filename}"}

```

Isječak koda 27: documents.py

Sljedeća API ruta omogućuje slanje e-mail poruke s kontakt forme. Korisnik ispunjava ime, email, broj telefona, adresu i poruku, a backend zatim šalje e-mail na zadanu adresu koristeći Gmail SMTP. Podaci se šalju putem SSL-a, a konfiguracija dolazi iz .env datoteke.

```
1 from fastapi import APIRouter
2 from pydantic import BaseModel, EmailStr
3 import smtplib
4 from email.mime.text import MIMEText
5 from email.utils import formataddr
6 from dotenv import load_dotenv
7 import os
8
9 load_dotenv()
10
11 router = APIRouter()
12
13 class ContactForm(BaseModel):
14     fullName: str
15     phone: str
16     email: EmailStr
17     address: str
18     description: str
19
20 @router.post("/send-email")
21 def send_email(data: ContactForm):
22     sender_email = os.getenv("GMAIL_USER")
23     sender_password = os.getenv("GMAIL_PASSWORD")
24     receiver_email = os.getenv("GMAIL_RECEIVER") or sender_email
25
26     msg_content = f"""
27     Nova poruka s kontakt forme:
28
29     Ime i prezime: {data.fullName}
30     Telefon: {data.phone}
31     E-mail: {data.email}
32     Adresa: {data.address}
33     Poruka:
34     {data.description}
35     """
36
37     msg = MIMEText(msg_content, "plain", "utf-8")
38     msg["Subject"] = "Nova poruka s web stranice"
39     msg["From"] = formataddr(("Web kontakt forma", sender_email))
40     msg["To"] = receiver_email
41     msg["Reply-To"] = data.email
42
43     try:
44         with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
45             server.login(sender_email, sender_password)
46             server.sendmail(sender_email, receiver_email, msg.as_string())
47         return {"message": "Email je uspješno poslan."}
48     except Exception as e:
49         print("Greška prilikom slanja emaila:", e)
50         return {"error": "Neuspješno slanje emaila. Provjerite postavke."}
```

Isječak koda 28: email.py

Sljedeća ruta omogućuje prikaz pregleda sadržaja .json datoteke s diska. Prvo provjerava postoji li tražena datoteka unutar mape Scraper/data, zatim provjerava je li to .json

datoteka. Ako sve prođe, učitava prvih 10 stavki (ako je riječ o listi) ili prvih 10 ključeva (ako je riječ o riječniku) i vraća ih kao preview. U slučaju greške (nepostojeća datoteka ili pogrešan format) vraća se odgovarajuća HTTP greška.

```
1 from fastapi import APIRouter, Query
2 from fastapi.responses import JSONResponse
3 import os
4 import json
5
6 router = APIRouter()
7
8 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
9 DOCUMENTS_DIR = os.path.abspath(os.path.join(BASE_DIR, "../Scraper/data"))
10
11 @router.get("/preview-document")
12 async def preview_document(filename: str = Query(...)):
13     try:
14         filepath = os.path.join(DOCUMENTS_DIR, filename)
15         if not os.path.isfile(filepath):
16             return JSONResponse(status_code=404, content={"error": "File not
17                 ↳ found."})
18
19         if not filename.endswith(".json"):
20             return JSONResponse(status_code=400, content={"error": "Only JSON
21                 ↳ preview is supported."})
22
23         with open(filepath, "r", encoding="utf-8") as f:
24             data = json.load(f)
25
26         preview = data
27         if isinstance(data, list):
28             preview = data[:10]
29         elif isinstance(data, dict):
30             preview = {k: data[k] for k in list(data)[:10]}
31
32         return {"preview": preview}
33
34     except Exception as e:
35         return JSONResponse(status_code=500, content={"error": str(e)})
```

Isječak koda 29: Preview.py

Ova API ruta vraća statistiku o trenutno učitanim dokumentima u bazi. Prvo dohvaća sve dokumente i filtrira one koji imaju metapodatke o vremenu učitavanja i izvoru. Ako postoji barem jedan takav dokument, određuje se zadnje učitani dokument prema vremenu. Također se izračunava ukupan broj dokumenata (po različitim source vrijednostima) i broj svih chunkova. Rezultat se vraća kao JSON s tri informacije: broj dokumenta, broj chunkova i naziv posljednje dodane datoteke.

```
1 from fastapi import APIRouter
2 from services.vectorstore import load_vectorstore
3 from datetime import datetime
4
5 router = APIRouter()
6
7 @router.get("/stats")
8 def get_stats():
9     db = load_vectorstore()
```

```

8 docs = list(db.docstore._dict.values())

9 docs_with_timestamps = [
10     doc for doc in docs if "uploaded_at" in doc.metadata and "source" in
        ↳ doc.metadata
11 ]

12 if docs_with_timestamps:
13     docs_sorted = sorted(docs_with_timestamps, key=lambda d:
        ↳ d.metadata["uploaded_at"])
14     last_added = docs_sorted[-1].metadata["source"]
15 else:
16     last_added = "--"

17 sources = {doc.metadata.get("source") for doc in docs if "source" in
        ↳ doc.metadata}

18 return {
19     "total_documents": len(sources),
20     "total_chunks": len(docs),
21     "last_added": last_added
22 }

```

Isječak koda 30: Stats.py

Zadnja API ruta omogućuje korisnicima da uploadaju PDF ili JSON dokumente putem forme. Primljeni dokumenti se obrađuju funkcije process documents , gdje se dijele na manje dijelove (chunkove). Ako su dokumenti uspješno obrađeni, poziva se get or create vectorstore kako bi se dokumenti dodali u vektorsku bazu. Na kraju se vraća poruka s brojem uspješno indeksiranih dijelova.

```

1 from fastapi import APIRouter, UploadFile, File, Form
2 from services.document_handler import process_documents
3 from services.vectorstore import get_or_create_vectorstore

4 router = APIRouter()

5 @router.post("/upload-documents")
6 async def upload_documents(
7     session_id: str = Form(...),
8     pdfs: list[UploadFile] = File(...)
9 ):
10     all_docs = await process_documents(pdfs)

11     if not all_docs:
12         return {"message": "No valid documents processed."}

13     get_or_create_vectorstore(all_docs)
14     return {"message": f"Successfully uploaded and indexed {len(all_docs)} chunks."}

```

Isječak koda 31: Upload.py

5.5. Prikaz rada aplikacije

Što se tiče frontenda, projekt je razvijan korištenjem Vite okoline, a pokretanje aplikacije omogućeno je korištenjem naredbe `npm run dev`.

Backend se pokreće lokalno pomoću Uvicorn ASGI servera, korištenjem naredbe `uvicorn main:app --reload` koja omogućuje automatsko ponovno učitavanje prilikom promjena u kodu.

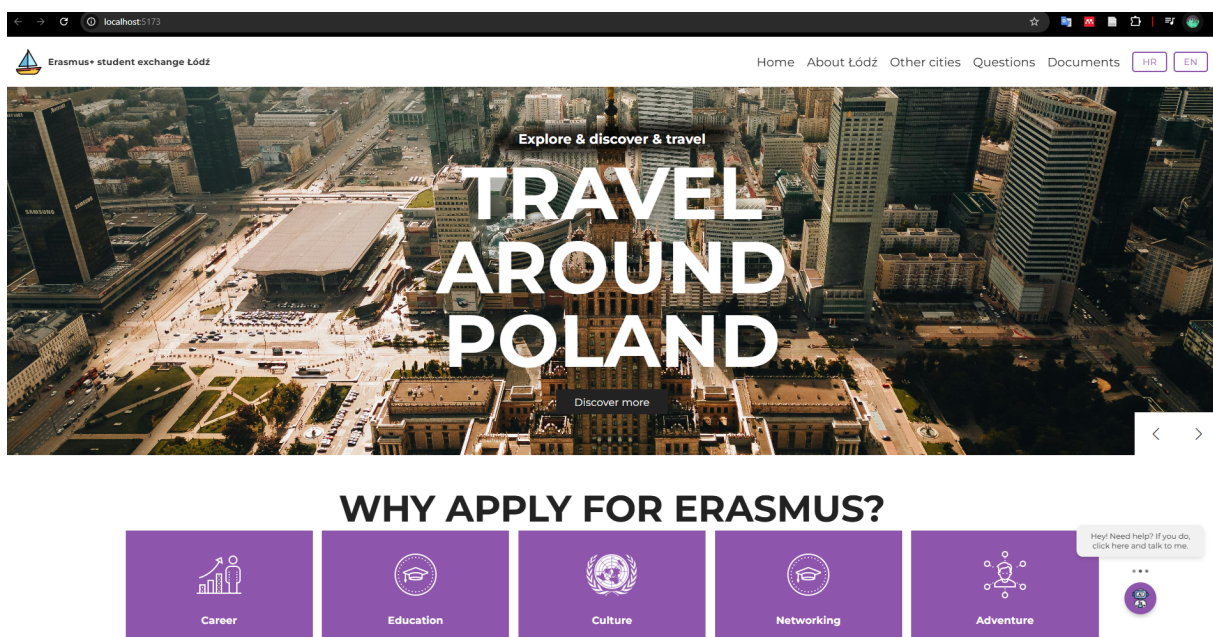
```
PS C:\Users\aleks\Radna površina\Website with a chatbot\Backend> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\aleks\\Radna površina\\Website with a chatbot\\Backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [10076] using StatReload
INFO: Started server process [17668]
INFO: Waiting for application startup.
INFO: Application startup complete.

PS C:\Users\aleks\Radna površina\Website with a chatbot\Frontend> npm run dev
> test@0.0.0 dev
> vite

VITE v6.2.2 ready in 329 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

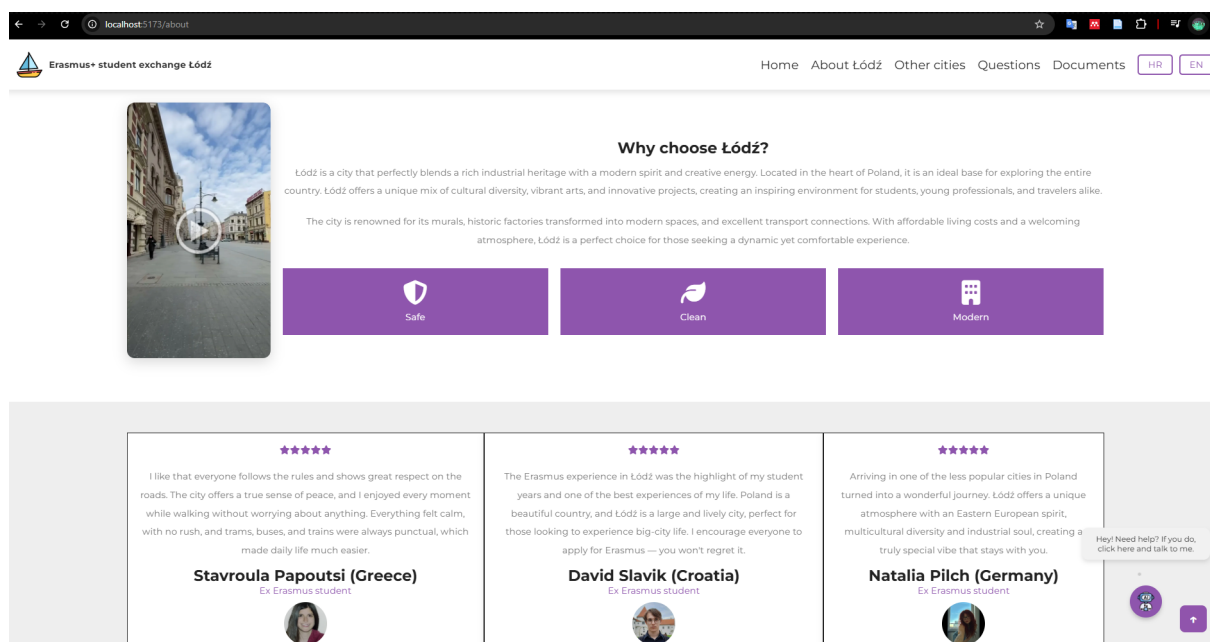
Slika 21: Pokretanje aplikacije kroz terminal

Sljedeća slika prikazuje početnu stranicu aplikacije na koju se dolazi nakon pokretanja aplikacije. Na početnoj stranici nalazi se nekoliko komponenta kao što su slider, motivacija, kratko o Łódźu i drugi gradovi. Ideja početne stranice je da korisniku pruži osnovne informacije i odvede korisnika na ostale stranice aplikacije.



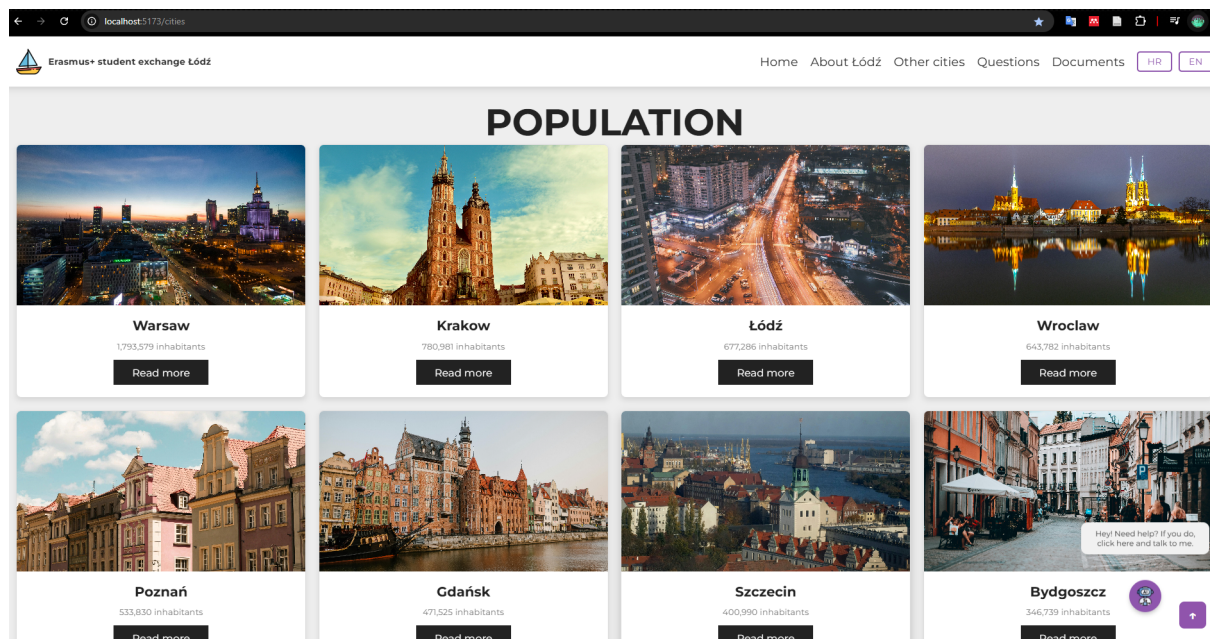
Slika 22: Početna stranica aplikacije

Sljedeća slika prikazuje stranicu O Łódźu koja sadrži dodatne informacije o gradu, uključujući iskustva bivših studenata koji su proveli Erasmus+ razmjenu u tom gradu.



Slika 23: O Łódźu stranica

Stranica drugi gradovi sadrži pregled drugih gradova u Poljskoj koje je moguće posjetiti. Ova stranica pruža korisnicima informacije u obliku slika i podataka kao što je broj stanovnika.



Slika 24: Drugi gradovi stranica

Sljedeća stranica namijenjena je za postavljanje pitanja od strane korisnika u slučaju da AI asistent ne pruži odgovor koji korisnik traži. Nakon slanja odgovora, vlasnik stranice dobiva email s informacijama prema definiranoj formi.

The screenshot shows a web browser at localhost:5173/question. The page has a header with the site logo and navigation links: Home, About Łódź, Other cities, Questions, Documents, HR, and EN. The main content area is titled 'If you have any questions or need assistance, feel free to reach out – I'm here to help.' It contains a form with the following fields: Full Name (John Doe), Phone ((123) 456-7890), Email (john.doe@example.com), Address (123 Main St, Cityville), and Description (Briefly describe your inquiry or issue). A 'Submit' button is at the bottom. A footer bar contains 'Quick links', 'Extra links', 'Contact info', and 'Follow me' sections. A small tooltip on the right says 'Hey! Need help? If you do, click here and talk to me.'

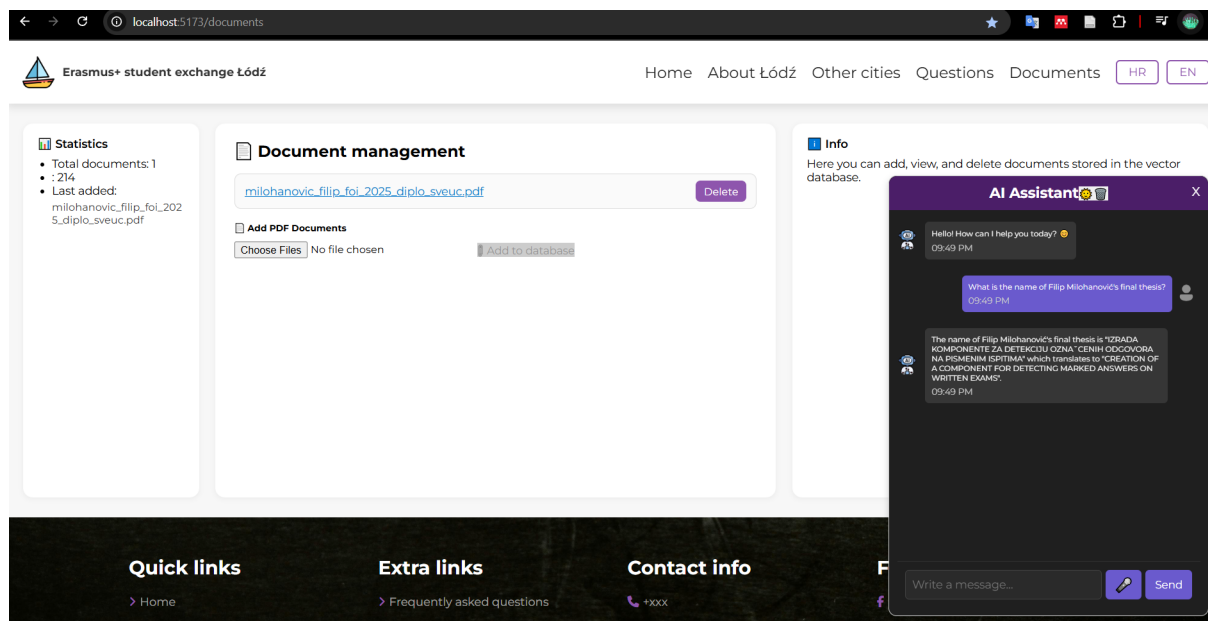
Slika 25: Forma za postavljanje pitanja stranica

Stranica za dokumente omogućuje pregled i upravljanje dokumentima koji se koriste za rad s asistentom. Prvotno ova stranica nije bila U početnoj ideji rada ali nakon što je započet rad s AI dijelom rada, pojavila se potreba za dodavanjem ove stranice kako bi se bolje vizualiziralo što se događa u pozadini rada asistenta.

The screenshot shows a web browser at localhost:5173/documents. The page layout includes a header with the site logo and navigation links. The main content area is divided into three sections: 'Statistics' (Total documents: 3, Last added: lodz_articles_en.json), 'Document management' (listing lodz_articles_en.json, lodz_attractions.pdf, and milohanovic_filip_foi_2025_diplo_svsuc.pdf with 'Delete' buttons), and 'Add PDF Documents' (with a 'Choose Files' button and an 'Add to database' button). An 'Info' section explains that users can add, view, and delete documents. An 'AI Assistant' chat window is open on the right, showing a conversation. The footer bar is identical to the previous screenshot.

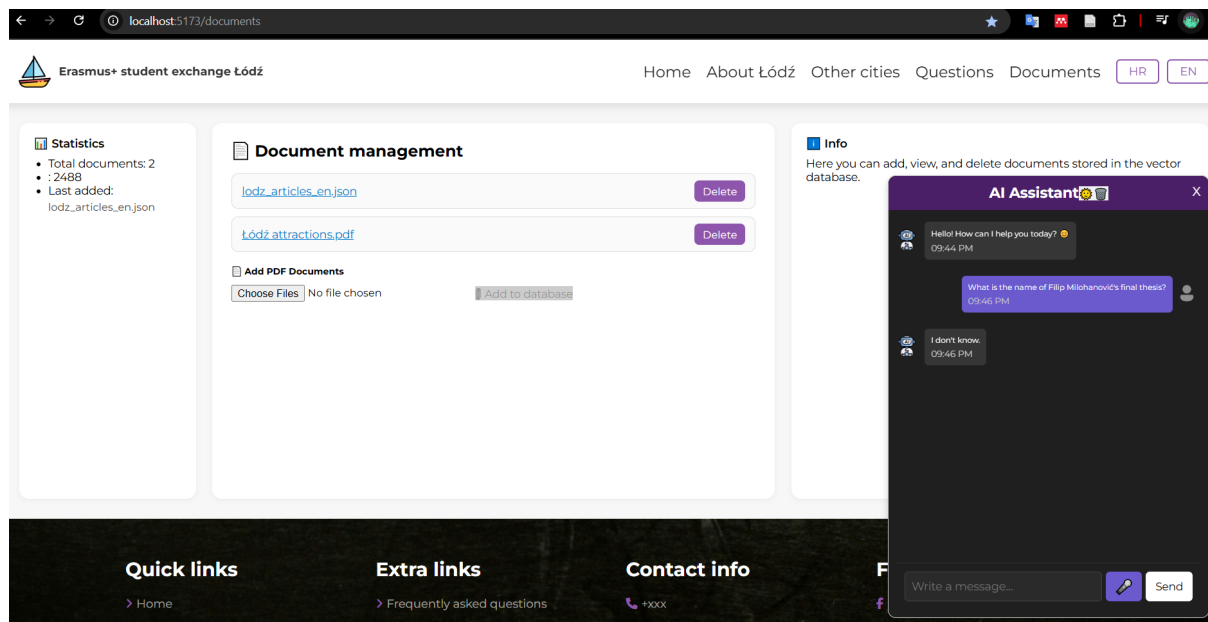
Slika 26: Stranica za pregled i upravljanje dokumentima

Na sljedećoj slici može se vidjeti kako funkcionira AI asistent kada je dokument u bazi podataka. Asistent u tom slučaju može odgovoriti na pitanje korisnika koristeći informacije iz dokumenta.



Slika 27: Testiranje asistenta kada je dokument u bazi

Na sljedećoj slici može se vidjeti kako AI asistent reagira kada dokument nije pronađen u bazi podataka.



Slika 28: Testiranje asistenta kad dokument nije u bazi

Kroz terminal može se vidjeti kako sustav radi u pozadini. Može se vidjeti koji je zahtjev poslan i kakav je status takvog zahtjeva.

```
response = qa_chain.run(question)
INFO:      127.0.0.1:16127 - "POST /ask HTTP/1.1" 200 OK
INFO:      127.0.0.1:16134 - "OPTIONS /delete-document?filename=babic_aleksa
ndar_foi_2023_predd_sveuc.pdf HTTP/1.1" 200 OK
INFO:      127.0.0.1:16134 - "DELETE /delete-document?filename=babic_aleksan
dar_foi_2023_predd_sveuc.pdf HTTP/1.1" 200 OK
INFO:      127.0.0.1:16134 - "GET /documents HTTP/1.1" 200 OK
INFO:      127.0.0.1:16135 - "GET /stats HTTP/1.1" 200 OK
INFO:      127.0.0.1:16135 - "POST /ask HTTP/1.1" 200 OK
```

Slika 29: Povratne informacije o radu aplikacije

5.6. Praćenje performansi i ograničenja korištenih alata

Na sljedećoj slici je prikazan dashboard s Groq platforme, konkretno za model llama-3.1-8b-instant. Prikazuju se grafovi s brojem zahtjeva i ukupnim brojem tokena u određenom vremenskom okviru. Ovo omogućuje korisnicima praćenje iskorištenosti modela u realnom vremenu te detekciju prekoračenja limita koji su označeni crvenim crtama. Prikazani vrhovi na grafu označavaju trenutke kada su slani upiti prema modelu. Osim toga moguće je pregledati i dnevnu/mjesečnu potrošnju u smislu financijskih sredstava. Isto tako kroz karticu Logs omogućen je uvid detaljniji opisa zahtjeva koji je stigao na Groq (kada je kreiran, koji model je korišten, koji API key, status code, kolko input tokena i output je potrošeno i od kog usera)



Slika 30: Groq platforma - pregled zahtjeva i potrošenih tokena

Ova slika prikazuje tablicu s ograničenjima korištenja za različite modele unutar Groq platforme. Za svaki model prikazani su maksimalni brojevi upita u minuti i danu, kao i ograničenja na broj tokena. Na primjer, model llamallama-3.1-8b-instant dopušta 30 zahtjeva u minuti, 14400 dnevno, i do 500000 tokena mjesečno. Ova tablica jasno prikazuje tehnička ograničenja koja korisnici moraju imati prilikom dizajniranja aplikacija koje koriste LLM modele u oblaku.

ID	REQUESTS PER MINUTE	REQUESTS PER DAY	TOKENS PER MINUTE	TOKENS PER DAY
allam-2-7b	30	7,000	6,000	500,000
compound-beta	15	200	70,000	(No limit)
compound-beta-mini	15	200	70,000	(No limit)
deepseek-r1-distill-llama-70b	30	1,000	6,000	100,000
gemma2-9b-it	30	14,400	15,000	500,000
llama-3.1-8b-instant	30	14,400	6,000	500,000
llama-3.3-70b-versatile	30	1,000	12,000	100,000
llama3-70b-8192	30	14,400	6,000	500,000
llama3-8b-8192	30	14,400	6,000	500,000
meta-llama/llama-4-maverick-17b-128e-instruct	30	1,000	6,000	500,000
meta-llama/llama-4-scout-17b-16e-instruct	30	1,000	30,000	500,000
meta-llama/llama-guard-4-12b	30	14,400	15,000	500,000

Slika 31: Groq limitacije

6. Zaključak

Za kraj, Erasmus se ne treba bojati jer je to prekasno iskustvo koje se pamti do kraja života. Svaki problem s kojim se suoči osoba, čini osobu jačom i neovisnijom. Boravak na jednom mjestu dulje vrijeme postat će čudan način života. Jednom Erasmus+, uvijek Erasmus+!

Poveznica na Github: **Poveznica**

Popis literature

- [1] Digital 360. „Are studies important in life? If yes, then why?” Pristupljeno: 10. svibanj 2025. adresa: <https://digital360india.medium.com/are-studies-important-in-life-if-yes-then-why-4583029b95e3>.
- [2] K. Bisio. „15 Reasons Why College is Important for Your Career.” Pristupljeno: 11. svibanj 2025. adresa: <https://www.msudenver.edu/15-reasons-why-college-is-important-for-your-career/>.
- [3] „What is Erasmus+? - Erasmus+.” Pristupljeno: 27. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/about-erasmus/what-is-erasmus>.
- [4] „Data on Erasmus+ in Croatia in 2023 - Erasmus+.” Pristupljeno: 10. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/factsheets/2023/croatia>.
- [5] „Studying abroad - Erasmus+.” Pristupljeno: 10. svibanj 2025. adresa: <https://erasmus-plus.ec.europa.eu/opportunities/opportunities-for-individuals/students/studying-abroad>.
- [6] Erasmus Generation Blog. „Six common Erasmus fears and how to deal with them.” Pristupljeno: 22. svibanj 2025. adresa: <https://blog.erasmusgeneration.org/six-common-erasmus-fears-and-how-deal-them>.
- [7] Erasmus Student Network AISBL. „Erasmus Careers - Feeling homesick.” Video. Objavljeno: 15. listopad 2024. Pristupljeno: 22. svibanj 2025. adresa: https://www.youtube.com/watch?v=TT__3yOkAdk.
- [8] J. Buchan. „Why you should add an AI assistant to your app or website.” Pristupljeno: 06. svibanj 2025., Zudu. adresa: <https://zudu.co.uk/blog/why-you-should-add-an-ai-assistant-to-your-app-or-website/>.
- [9] c2061348. „The difference between ChatBot, AI Assistant, Copilot, and AI Agent.” Pristupljeno: 10. svibanj 2025. adresa: <https://exomindset.co/the-difference-between-chatbot-ai-assistant-copilot-and-ai-agent/>.
- [10] E. Mixon. „What is Siri? | Definition from TechTarget.” Pristupljeno: 20. lipanj 2025. adresa: <https://www.techtarget.com/searchmobilecomputing/definition/Siri>.
- [11] „Create Virtual Agents with Copilot | Microsoft Copilot Studio.” Pristupljeno: 27. svibanj 2025. adresa: <https://www.microsoft.com/en-us/copilot/microsoft-copilot-studio>.

- [12] iaanw. „Quotas and limits - Microsoft Copilot Studio.” Pristupljeno: 27. svibanj 2025. adresa: <https://learn.microsoft.com/en-us/microsoft-copilot-studio/requirements-quotas>.
- [13] „Introducing Operator.” Pristupljeno: 10. svibanj 2025. adresa: <https://openai.com/index/introducing-operator/>.
- [14] Riley Brown. „TikTok video objavljen 24. siječnja 2025.” [Online video]. Pristupljeno: 4. lipanj 2025., TikTok. adresa: <https://www.tiktok.com/@rileybrown.ai/video/7463264903327976734>.
- [15] A. Connell. „50 Critical Chatbot Statistics You Need To Know In 2025.” Accessed online, pogledano 12. srpnja 2025. adresa: <https://adamconnell.me/chatbot-statistics/>.
- [16] „Gradescope | Save time grading.” Pristupljeno: 26. svibanj 2025. adresa: <https://www.gradescope.com/>.
- [17] P. Weber. „SmythOS - Intelligent Agents in Education.” Pristupljeno: 26. svibanj 2025. adresa: <https://smythos.com/ai-industry-solutions/education/intelligent-agents-in-education/>.
- [18] Amazon Web Services, Inc. „What is RAG? - Retrieval-Augmented Generation AI Explained - AWS.” Pristupljeno: 06. svibanj 2025. adresa: <https://aws.amazon.com/what-is/retrieval-augmented-generation/>.
- [19] Marina Danilevsky, *What is Retrieval-Augmented Generation (RAG)?* [Online video]. Pristupljeno: 6. svibanj 2025., IBM Technology, 8. 2023.
- [20] IBM Research. „What is retrieval-augmented generation (RAG)?” Pristupljeno: 06. svibanj 2025. adresa: <https://research.ibm.com/blog/retrieval-augmented-generation-RAG>.
- [21] R. W. G. Studio) MD (Custom AI. „This history of Retrieval-Augmented Generation in 3 minutes...!” Medium, pogledano 29. svibnja 2025. adresa: https://medium.com/@custom_aistudio/this-history-of-retrieval-augmented-generation-in-3-minutes-f7f07073599a.
- [22] Redis. „Introduction to Retrieval Augmented Generation (RAG).” Redis, pogledano 29. svibnja 2025. adresa: <https://redis.io/glossary/retrieval-augmented-generation/>.
- [23] Shaheryar. „Understanding the Key Components of RAG: Retriever and Generator.” Pristupljeno: 29. svibanj 2025. adresa: <https://dev.to/shaheryaryousaf/understanding-the-key-components-of-rag-retriever-and-generator-1a1j>.
- [24] S. Gupta, R. Ranjan i S. N. Singh, „A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions,” *arXiv preprint arXiv:2410.12837*, 10. 2024., Preuzeto: 3. listopada 2024. DOI: 10.48550/arXiv.2410.12837.
- [25] OpenAI. „GPT-4 Technical Report,” pogledano 26. srpnja 2025. adresa: <https://cdn.openai.com/papers/gpt-4.pdf>.

- [26] Monte Carlo Data. „RAG Vs Fine Tuning: How To Choose The Right Method,” pogledano 26. srpnja 2025. adresa: <https://www.montecarloata.com/blog-rag-vs-fine-tuning/>.
- [27] A. Siddharth. „A practical guide to RAG implementation: Part I | LinkedIn.” Pristupljeno: 04. lipanj 2025. adresa: <https://www.linkedin.com/pulse/practical-guide-rag-implementation-part-i-siddharth-asthana-bppbe/>.
- [28] Nishtha. „Top 7 RAG Use Cases and Applications to Explore in 2025.” Pristupljeno: 20. lipanj 2025., ProjectPro. adresa: <https://www.projectpro.io/article/rag-use-cases-and-applications/1059>.
- [29] Aim. Labs i P. Anand. „Top Problems with RAG systems and ways to mitigate them - AIMon Labs.” Pristupljeno: 2025-06-04. adresa: https://www.aimon.ai/posts/top%5C_problems%5C_with%5C_rag%5C_systems%5C_and%5C_ways%5C_to%5C_mitigate%5C_them.
- [30] K2View. „RAG Hallucination: What is It and How to Avoid It,” pogledano 29. svibnja 2025. adresa: <https://www.k2view.com/blog/rag-hallucination/>.
- [31] C. Sharma, „Retrieval-Augmented Generation: A Comprehensive Survey of Architectures, Enhancements, and Robustness Frontiers,” *arXiv preprint arXiv:2506.00054*, 2025., Pristupljeno: 26. srpanj 2025. Preprint pod recenzijom u ACM TOIS.
- [32] I. Novogroder. „Top 9 RAG Tools to Boost Your LLM Workflows.” Pristupljeno: 2025-06-02. adresa: <https://lakefs.io/blog/rag-tools/>.
- [33] S. Aquino. „An Introduction to Vector Databases - Qdrant.” Pristupljeno: 2. srpnja 2025. adresa: <https://qdrant.tech/articles/what-is-a-vector-database/>.
- [34] V. Baghel. „13 Best Vector Databases for AI-Powered Solutions.” Openxcell. Pristupljeno: 04. lipanj 2025. adresa: <https://www.openxcell.com/blog/best-vector-databases/>.
- [35] M. Vermeir. „What are large language models (LLMs)?” Accessed: Jul. 11, 2025, ABBYY. adresa: <https://www.abbyy.com/blog/large-language-models/>.
- [36] M. Ozkaya. „LLM Providers: OpenAI, Meta AI, Anthropic, Hugging Face, Microsoft, Google, and Mistral AI.” Medium. Pristupljeno: 04. lipanj 2025. adresa: <https://mehmetozkaya.medium.com/llm-providers-openai-meta-ai-anthropic-hugging-face-microsoft-google-and-mistral-ai-46ad8c027f6b>.
- [37] „What Is React? [Easily Explained].” Pristupljeno: 11. svibanj 2025. adresa: <https://www.simplilearn.com/tutorials/reactjs-tutorial/what-is-reactjs>.
- [38] A. Gillis i R. Awati. „What is SMTP (Simple Mail Transfer Protocol)?” Pristupljeno: 08. srpanj 2025. adresa: <https://www.techtarget.com/whatis/definition/SMTP-Simple-Mail-Transfer-Protocol>.
- [39] Obafemi. „Building a Website with Python, FastAPI, and Streamlit.” Pristupljeno: 7. srpnja 2025., pogledano 7. srpnja 2025. adresa: <https://medium.com/@obaff/building-a-website-with-python-fastapi-and-streamlit-418f48c41af2>.

Popis slika

1.	Primjer copilota kreiranog u Microsoft Copilot Studiu	10
2.	Adaptive cards	10
3.	Globalno tržište chatbotova; Izvor	12
4.	Ocjenjivanje zadatka - Gradescope [16]	14
5.	Popularnost RAG-a; Izvor	17
6.	Konceptualni tok korištenja RAG-a s LLM-ovima [18]	19
7.	Fino podešavanje proces (generalno); Izvor	22
8.	Višeskočno zaključivanje - primjer; Izvor	27
9.	Sažetak dostupnih alata i biblioteka za razvoj RAG sustava; Izvor	29
10.	Vektorska baza - primjer; Izvor	32
11.	Popularnost LLM-ova u rješavanju matematičkih problema za 2024 godinu; Izvor	34
12.	React komponenta	35
13.	Proces dostavljanja emaila; Izvor	36
14.	Interaktivna dokumentacija (OpenAPI) u Swagger stilu	37
15.	ReDoc prikaz dokumentacije	37
16.	Struktura praktičnog dijela rada	38
17.	Struktura scraper foldera	40
18.	Struktura frontend dijela	44
19.	Struktura backend dijela	49
20.	Sadržaj env datoteke	50
21.	Pokretanje aplikacije kroz terminal	59
22.	Početna stranica aplikacije	59
23.	O Łódźu stranica	60

24. Drugi gradovi stranica	60
25. Forma za postavljanje pitanja stranica	61
26. Stranica za pregled i upravljanje dokumentima	61
27. Testiranje asistenta kada je dokument u bazi	62
28. Testiranje asistenta kad dokument nije u bazi	62
29. Povratne informacije o radu aplikacije	63
30. Groq platforma - pregled zahtjeva i potrošenih tokena	64
31. Groq limitacije	64

Popis tablica

1.	Razlika između pojmova [9]	7
2.	Usporedba RAG pristupa i finog podešavanja modela [26]	23
3.	Prednosti RAG-a [18]	23
4.	Usporedba OLTP, OLAP i vektorskih baza podataka [33]	32
5.	Struktura stranica i njihovih komponenti	39
6.	Korišteni izvori podataka i alati za obradu	40

Popis isječaka koda

1.	settings.json	38
2.	Scraper 1/6	41
3.	Scraper 2/6	41
4.	Scraper 3/6	42
5.	Scraper 4/6	42
6.	Scraper 5/6	42
7.	Scraper 6/6	43
8.	Chat.jsx	44
9.	Contact.jsx	45
10.	Document.jsx 1/5	45
11.	Document.jsx 2/5	45
12.	Document.jsx 3/5	45
13.	Document.jsx 4/5	46
14.	Document.jsx 5/5	46
15.	About.jsx komponenta	47
16.	LanguageContext.jsx	48
17.	Stranica Home	48
18.	App.jsx	49
19.	Main.py	50
20.	Vectorstore.py 1/4	51
21.	Vectorstore.py 2/4	51
22.	Vectorstore.py 3/4	52
23.	Vectorstore.py 4/4	52
24.	qa.py	53

25.	<code>Document_handler.py</code>	54
26.	<code>ask.py</code>	55
27.	<code>documents.py</code>	56
28.	<code>email.py</code>	56
29.	<code>Preview.py</code>	57
30.	<code>Stats.py</code>	58
31.	<code>Upload.py</code>	58